

**ĐẠI HỌC QUỐC GIA HÀ NỘI  
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



**Vũ Tùng Lâm**

**ỨNG DỤNG HỌC MÁY TRONG PHÁT HIỆN MÃ ĐỘC  
PE TRÊN WINDOWS**

**KHOÁ LUẬN TỐT NGHIỆP**

**Ngành: Công nghệ thông tin**

**HÀ NỘI - 2026**

**ĐẠI HỌC QUỐC GIA HÀ NỘI  
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



**VŨ TÙNG LÂM**

**ỨNG DỤNG HỌC MÁY TRONG PHÁT HIỆN MÃ ĐỘC  
PE TRÊN WINDOWS**

**KHOÁ LUẬN TỐT NGHIỆP**

**Ngành: Công nghệ thông tin**

**Cán bộ hướng dẫn: TS. Lê Đình Thanh**

**HÀ NỘI - 2026**

**VIETNAM NATIONAL UNIVERSITY, HANOI  
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



**Vu Tung Lam**

**PE MALWARE DETECTION ON WINDOWS USING MACHINE  
LEARNING TECHNIQUES**

**BACHELOR'S THESIS**

**Major: Information Technology**

**Supervisor: Dr. Le Dinh Thanh**

**HANOI - 2026**

## Lời cảm ơn

Trước hết, em xin được gửi lời cảm ơn chân thành và sâu sắc tới thầy Lê Đình Thanh, người đã trực tiếp hướng dẫn, hỗ trợ và đồng hành cùng em trong suốt quá trình thực hiện khóa luận tốt nghiệp này. Những góp ý chuyên môn, các buổi trao đổi tận tình cùng sự động viên của thầy đã giúp em định hướng rõ ràng hơn trong quá trình nghiên cứu, đồng thời là nguồn động lực quan trọng để em vượt qua khó khăn và hoàn thành khóa luận của mình.

Em cũng xin bày tỏ lòng biết ơn tới toàn thể các thầy, cô giáo tại Trường Đại học Công nghệ nói chung, và các thầy cô giáo của khoa Công nghệ thông tin nói riêng, vì đã tận tâm giảng dạy, truyền đạt kiến thức và hỗ trợ em trong suốt những năm tháng học tập tại trường. Những kiến thức, kỹ năng và kinh nghiệm mà em tiếp thu được từ các thầy cô chính là nền tảng quý báu, giúp em có đủ hành trang để hoàn thành khóa luận cũng như tự tin hơn trên con đường học tập và sự nghiệp sau này.

Xin cảm ơn tập thể sinh viên Trường Đại học Công nghệ, đặc biệt là các bạn sinh viên K67I-CS4, K67I-IS, và K67I-CN. Các bạn chính là những người đồng hành tuyệt vời, cùng tôi vượt qua những khó khăn, chia sẻ niềm vui và tạo nên những kỷ niệm đáng nhớ trong suốt chặng đường đại học đầy ý nghĩa.

Cuối cùng và cũng là nguồn động viên lớn nhất, con xin bày tỏ lòng biết ơn sâu sắc tới bố mẹ và gia đình, những người luôn lắng nghe, thấu hiểu và làm điểm tựa vững chắc không chỉ những năm tháng đại học mà cả trong những năm tháng cuộc đời.

Em xin chân thành cảm ơn!

## Lời cam đoan

Tôi là Vũ Tùng Lâm, sinh viên lớp K67-I-CS4 Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội.

Tôi xin cam đoan khóa luận tốt nghiệp ***Ứng dụng học máy trong phát hiện mã độc PE trên Windows*** là do tôi tự tìm hiểu, nghiên cứu và phát triển dưới sự hướng dẫn của TS. Lê Đình Thanh, không sao chép hay sử dụng bất kỳ phần nào từ các công trình nghiên cứu của người khác mà không trích dẫn nguồn theo quy định. Tất cả các tài liệu tham khảo đều đã được liệt kê đầy đủ trong phần Tài liệu tham khảo của khóa luận.

Trong quá trình thực hiện khóa luận, tôi có sử dụng các công cụ hỗ trợ trí tuệ nhân tạo như ChatGPT và Claude nhằm hỗ trợ kiểm tra, chỉnh sửa ngữ pháp cho toàn bộ nội dung khóa luận, hỗ trợ tham khảo trong phần tổng quan nghiên cứu (*literature review*) và gợi ý cú pháp trong quá trình lập trình. Tuy nhiên, toàn bộ ý tưởng, nội dung cốt lõi, quá trình nghiên cứu, triển khai và lập luận học thuật đều do tôi tự thực hiện và chịu trách nhiệm.

Tôi xin hoàn toàn chịu trách nhiệm về tính trung thực và tính chính xác của toàn bộ nội dung trong khóa luận này.

Hà Nội, ngày 22 tháng 05 năm 2026

Sinh viên

Vũ Tùng Lâm

## Tóm tắt

# Tóm tắt

Khóa luận này nghiên cứu ứng dụng học máy trong phát hiện mã độc định dạng PE (Portable Executable) trên hệ điều hành Windows, với hai đóng góp chính: đánh giá thực nghiệm hiệu quả của kỹ thuật trích xuất năng lực mã độc (capability extraction) trong pipeline học máy, và xây dựng hệ thống bảo vệ endpoint hoạt động được trên Windows.

Về mặt nghiên cứu, khóa luận đề xuất và so sánh ba pipeline học máy dựa trên mô hình LightGBM, sử dụng tập dữ liệu EMBER2024. Ba pipeline khác nhau ở cách xây dựng vector đặc trưng đầu vào: chỉ dùng đặc trưng EMBER2024 (baseline), bổ sung đặc trưng năng lực từ công cụ Capa dạng one-hot, và bổ sung đặc trưng Capa đã giảm chiều bằng TruncatedSVD. Kết quả thực nghiệm trên 450.000 mẫu huấn luyện và 50.000 mẫu kiểm tra cho thấy pipeline kết hợp EMBER2024 với Capa TruncatedSVD-256 đạt hiệu quả tốt nhất với Partial AUC ( $\text{FPR} \leq 0,5\%$ ) = 0,8850 và Full AUC = 0,9926, cải thiện so với baseline lần lượt là 0,34% và 0,04%. Kết quả này cho thấy thông tin ngữ nghĩa hành vi từ Capa mang lại giá trị phân biệt bổ sung thực chất so với đặc trưng cấu trúc PE thông thường.

Về mặt công nghệ, khóa luận thiết kế và hiện thực hóa kiến trúc hệ thống gồm năm thành phần: module trích xuất đặc trưng hiệu năng cao viết bằng C++ (nhanh hơn từ 12 đến 20 lần so với thuật toán gốc được viết bằng Python của dự án EMBER2024), kernel driver phát hiện tiến trình thực thi dựa trên KMDF, dịch vụ nền tích hợp mô hình học máy, giao diện người dùng với thông báo thời gian thực, và dịch vụ phân tích sâu trực tuyến tích hợp Capa và SHAP. Hệ thống cho phép phát hiện và chặn mã độc ngay tại thời điểm thực thi tại endpoint, đồng thời cung cấp khả năng phân tích sâu và giải thích quyết định của mô hình thông qua dịch vụ phân tích chuyên sâu trực tuyến.

**Từ khóa:** mã độc, học máy, EMBER2024, trích xuất năng lực, Windows driver.

**GitHub repository:** [https://github.com/laam-egg/KLTN\\_22028235](https://github.com/laam-egg/KLTN_22028235)

# Abstract

This thesis investigates the application of machine learning to the detection of Portable Executable (PE) malware on the Windows operating system, with two main contributions: an empirical evaluation of capability extraction as a feature engineering technique in machine learning pipelines, and the design and implementation of a functional endpoint protection system on Windows.

On the research side, three LightGBM-based machine learning pipelines are proposed and compared using the EMBER2024 dataset. The pipelines differ in how their input feature vectors are constructed: using only EMBER2024 features (baseline), augmenting with Capa-derived capability features encoded as one-hot vectors, and augmenting with Capa features reduced via TruncatedSVD. Experiments conducted on 450,000 training samples and 50,000 test samples show that the pipeline combining EMBER2024 with Capa TruncatedSVD-256 achieves the best performance, with a Partial AUC ( $\text{FPR} \leq 0.5\%$ ) of 0.8850 and a Full AUC of 0.9926, improving over the baseline by 0.34% and 0.04% respectively. These results demonstrate that the behavioral semantic information provided by Capa contributes meaningful discriminative value beyond conventional PE structural features.

On the engineering side, the thesis designs and implements a five- component system architecture: a high-performance C++ feature extraction module (12 to 20 times faster than the original Python implementation of the EMBER2024 project), a KMDF-based kernel driver for process execution monitoring, a Windows background service integrating the machine learning model, a real-time notification user interface, and an online deep analysis service incorporating Capa and SHAP. The system is capable of detecting and blocking malware at the moment of execution on the endpoint, while also providing in-depth analysis and model decision explanations through the online deep analysis service.

**Keywords:** malware detection, machine learning, EMBER2024, capability extraction, Windows driver.

**GitHub repository:** [https://github.com/laam-egg/KLTN\\_22028235](https://github.com/laam-egg/KLTN_22028235)



# Mục lục

Lời cảm ơn

Lời cam đoan i

Tóm tắt ii

Tóm tắt iii

Abstract iv

Mục lục v

Danh sách hình vẽ viii

Danh sách bảng ix

Danh mục các từ viết tắt x

Mở đầu 1

Đặt vấn đề . . . . . 1

Mô tả bài toán . . . . . 4

Mục tiêu và phạm vi nghiên cứu của khóa luận . . . . . 6

Cấu trúc khóa luận . . . . . 7

**Chương 1 Cơ sở lý thuyết 9**

1.1 Tổng quan về mã độc PE trên hệ điều hành Windows . . . . . 9

1.1.1 Tổng quan về mã độc (malware) . . . . . 9

1.1.2 Tổng quan về các phương pháp phát hiện và ngăn chặn mã độc 11

1.1.3 Tổng quan về x86/amd64 Assembly . . . . . 12

1.1.4 Tổng quan về định dạng PE (Portable Executable) trên Windows 15

1.1.5 Tổng quan về các phương pháp phân tích mã độc PE . . . . . 17

1.2 Tổng quan về ứng dụng học máy trong phát hiện mã độc . . . . . 20

|                                                                                  |                                                                                                    |           |
|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|-----------|
| 1.2.1                                                                            | Các hướng tiếp cận phổ biến trong phát hiện mã độc bằng học máy . . . . .                          | 21        |
| 1.2.2                                                                            | Trích xuất đặc trưng từ file PE . . . . .                                                          | 22        |
| 1.2.3                                                                            | Một số mô hình học máy tiêu biểu trong phát hiện mã độc PE                                         | 26        |
| 1.2.4                                                                            | Mô hình LightGBM . . . . .                                                                         | 27        |
| 1.3                                                                              | Các công trình nghiên cứu liên quan . . . . .                                                      | 28        |
| 1.3.1                                                                            | Các tập dữ liệu file PE phục vụ huấn luyện mô hình học máy phát hiện mã độc . . . . .              | 28        |
| 1.3.2                                                                            | Các nghiên cứu về ứng dụng học máy trong phân tích tĩnh file thực thi nhị phân . . . . .           | 29        |
| 1.3.3                                                                            | Các nghiên cứu về trích xuất năng lực (capability extraction) của file thực thi nhị phân . . . . . | 30        |
| 1.3.4                                                                            | Các nghiên cứu về lý giải quyết định của mô hình (Explainable AI - xAI) . . . . .                  | 31        |
| 1.4                                                                              | Công nghệ sử dụng . . . . .                                                                        | 32        |
| 1.4.1                                                                            | Ngôn ngữ lập trình và công cụ xây dựng . . . . .                                                   | 32        |
| 1.4.2                                                                            | Lập trình hệ thống Windows . . . . .                                                               | 33        |
| 1.4.3                                                                            | Phát triển dịch vụ phân tích . . . . .                                                             | 33        |
| 1.4.4                                                                            | Thư viện học máy và phân tích mã độc . . . . .                                                     | 34        |
| <b>Chương 2 Thiết kế và đánh giá các pipeline học máy trong phát hiện mã độc</b> |                                                                                                    | <b>36</b> |
| 2.1                                                                              | Tổng quan kiến trúc giải pháp . . . . .                                                            | 36        |
| 2.2                                                                              | Thiết kế các pipeline học máy . . . . .                                                            | 37        |
| 2.2.1                                                                            | Tập dữ liệu EMBER2024 . . . . .                                                                    | 37        |
| 2.2.2                                                                            | Pipeline A: EMBER2024 thuần . . . . .                                                              | 38        |
| 2.2.3                                                                            | Pipeline B: EMBER2024 kết hợp Capa multi-hot . . . . .                                             | 38        |
| 2.2.4                                                                            | Pipeline C: EMBER2024 kết hợp Capa TruncatedSVD . . . . .                                          | 40        |
| 2.2.5                                                                            | Cấu hình huấn luyện . . . . .                                                                      | 42        |
| 2.3                                                                              | Đánh giá và so sánh các pipeline . . . . .                                                         | 42        |
| 2.3.1                                                                            | Lựa chọn độ đo đánh giá . . . . .                                                                  | 42        |
| 2.3.2                                                                            | Kết quả thực nghiệm . . . . .                                                                      | 43        |
| 2.3.3                                                                            | Nhận xét kết quả . . . . .                                                                         | 43        |
| 2.3.4                                                                            | Thảo luận và lựa chọn pipeline triển khai . . . . .                                                | 44        |

|                                                                                                              |           |
|--------------------------------------------------------------------------------------------------------------|-----------|
| <b>Chương 3 Phát triển dịch vụ phát hiện mã độc trên máy endpoint<br/>và dịch vụ phân tích mã độc online</b> | <b>46</b> |
| 3.1 Phân tích yêu cầu hệ thống . . . . .                                                                     | 46        |
| 3.1.1 Yêu cầu chức năng . . . . .                                                                            | 46        |
| 3.1.2 Yêu cầu phi chức năng . . . . .                                                                        | 47        |
| 3.1.3 Tổng quan các thành phần hệ thống . . . . .                                                            | 48        |
| 3.2 Thiết kế hệ thống chi tiết . . . . .                                                                     | 48        |
| 3.2.1 Module trích xuất đặc trưng (ember2024-fe-cpp) . . . . .                                               | 48        |
| 3.2.2 Module driver (pmd-driver) . . . . .                                                                   | 50        |
| 3.2.3 Module dịch vụ nền (pmd-service) . . . . .                                                             | 51        |
| 3.2.4 Module giao diện người dùng (pmd-ui) . . . . .                                                         | 52        |
| 3.2.5 Module phân tích sâu (deep-analysis-server) . . . . .                                                  | 52        |
| 3.3 Cài đặt và kiểm thử . . . . .                                                                            | 55        |
| 3.3.1 Môi trường cài đặt . . . . .                                                                           | 55        |
| 3.3.2 Mô tả phần tự cài đặt và phần tận dụng . . . . .                                                       | 55        |
| 3.3.3 Kịch bản kiểm thử end-to-end . . . . .                                                                 | 55        |
| 3.3.4 Kết quả kiểm thử và đánh giá hiệu năng . . . . .                                                       | 55        |
| <b>Kết luận và hướng phát triển</b>                                                                          | <b>60</b> |
| <b>Tài liệu tham khảo</b>                                                                                    | <b>62</b> |

# Danh sách hình vẽ

|     |                                                                                                                 |    |
|-----|-----------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Các pipeline học máy được sử dụng trong phát hiện mã độc PE. . .                                                | 45 |
| 3.1 | Kiến trúc tổng thể của hệ thống phát hiện mã độc trên máy endpoint.                                             | 49 |
| 3.2 | Sequence diagram thể hiện luồng quét file PE tại máy endpoint . . .                                             | 50 |
| 3.3 | Giao diện chính của pmd-ui hiển thị lịch sử các lần phát hiện mã độc.                                           | 53 |
| 3.4 | Giao diện của pmd-ui gợi ý gửi mã độc lên dịch vụ phân tích chuyên sâu (deep-analysis-server). . . . .          | 54 |
| 3.5 | Giao diện web của deep-analysis-server, cho phép người dùng upload file thử công. . . . .                       | 55 |
| 3.6 | Kết quả TC2: tiến trình bị terminate và popup thông báo trên pmd-ui.                                            | 56 |
| 3.7 | Kết quả TC4: giao diện deep-analysis-server hiển thị kết quả phân loại và danh sách đóng góp đặc trưng. . . . . | 59 |

# Danh sách bảng

|     |                                                                                           |    |
|-----|-------------------------------------------------------------------------------------------|----|
| 1.1 | Các calling convention phổ biến trên Windows . . . . .                                    | 35 |
| 2.1 | Kết quả so sánh các pipeline theo ROC-AUC (validation) và AUC<br>(test set). . . . .      | 43 |
| 3.1 | Các thành phần chính của hệ thống . . . . .                                               | 48 |
| 3.2 | So sánh hiệu năng giữa hai bộ trích xuất đặc trưng EMBER2024 -<br>Python và C++ . . . . . | 49 |
| 3.3 | Môi trường cài đặt và kiểm thử . . . . .                                                  | 56 |
| 3.4 | Phân tách phần tự cài đặt và phần tận dụng . . . . .                                      | 57 |
| 3.5 | Kịch bản kiểm thử end-to-end . . . . .                                                    | 58 |
| 3.6 | Kết quả kiểm thử end-to-end . . . . .                                                     | 59 |

# Danh mục các từ viết tắt

| Từ viết tắt | Cụm từ đầy đủ                                         | Ý nghĩa / Mô tả                                                                 |
|-------------|-------------------------------------------------------|---------------------------------------------------------------------------------|
| API         | Application Programming Interface                     | Tập hợp giao thức cho phép các phần mềm giao tiếp với nhau                      |
| APT         | Advanced Persistent Threat                            | Chiến dịch tấn công có chủ đích                                                 |
| ASLR        | Address Space Layout Randomization                    | Cơ chế bảo mật ngẫu nhiên hóa vị trí vùng nhớ khi tải tiến trình                |
| ATT&CK      | Adversarial Tactics, Techniques, and Common Knowledge | Khung phân loại chiến thuật và kỹ thuật tấn công mạng do MITRE phát triển       |
| AUC         | Area Under the Curve                                  | Chỉ số đánh giá mô hình phân loại, đo diện tích dưới đường cong ROC             |
| AV          | Antivirus                                             | Phần mềm phát hiện và loại bỏ mã độc trên máy tính                              |
| COFF        | Common Object File Format                             | Định dạng file nhị phân chuẩn dùng cho các module thực thi trên Windows         |
| DEP         | Data Execution Prevention                             | Cơ chế bảo mật ngăn thực thi mã từ vùng nhớ chỉ dành cho dữ liệu                |
| DLL         | Dynamic-Link Library                                  | File thư viện chứa mã và tài nguyên dùng chung giữa các tiến trình Windows      |
| EDR         | Endpoint Detection and Response                       | Giải pháp giám sát, phát hiện và phản ứng với mối đe dọa trên thiết bị đầu cuối |
| EMBER       | Endgame Malware Benchmark for Research                | Tập dữ liệu benchmark mã nguồn mở phục vụ nghiên cứu phát hiện mã độc PE        |

| Từ viết tắt | Cụm từ đầy đủ                | Ý nghĩa / Mô tả                                                            |
|-------------|------------------------------|----------------------------------------------------------------------------|
| EV          | Extended Validation          | Loại chứng chỉ SSL/TLS yêu cầu xác minh danh tính tổ chức nghiêm ngặt      |
| FPR         | False Positive Rate          | Tỷ lệ mẫu lành tính bị mô hình phân loại nhầm thành mã độc                 |
| GUI         | Graphical User Interface     | Giao diện người dùng tương tác thông qua các thành phần đồ họa             |
| IAT         | Import Address Table         | Bảng lưu địa chỉ các hàm được nạp động từ DLL khi thực thi                 |
| IOCTL       | Input/Output Control         | Cơ chế gửi lệnh điều khiển trực tiếp từ ứng dụng xuống driver kernel       |
| IPC         | Inter-Process Communication  | Giao tiếp liên tiến trình                                                  |
| KMDF        | Kernel-Mode Driver Framework | Framework của Microsoft hỗ trợ phát triển driver hoạt động ở chế độ nhân   |
| LGBM        | LightGBM                     | Thuật toán gradient boosting hiệu năng cao, tối ưu cho dữ liệu lớn         |
| MBC         | Malware Behavior Catalog     | Danh mục chuẩn hóa các hành vi của mã độc phục vụ phân tích và phân loại   |
| ML          | Machine Learning             | Lĩnh vực AI cho phép máy tính học từ dữ liệu mà không lập trình tường minh |
| PCA         | Principal Component Analysis | Kỹ thuật giảm chiều dữ liệu bằng cách chiếu lên các thành phần chính       |
| PE          | Portable Executable          | Định dạng file thực thi và thư viện chuẩn trên hệ điều hành Windows        |

| Từ viết tắt | Cụm từ đầy đủ                       | Ý nghĩa / Mô tả                                                                           |
|-------------|-------------------------------------|-------------------------------------------------------------------------------------------|
| PID         | Process Identifier                  | Số định danh duy nhất do hệ điều hành cấp cho mỗi tiến trình đang chạy                    |
| ROC         | Receiver Operating Characteristic   | Đồ thị biểu diễn sự đánh đổi giữa TPR và FPR của mô hình phân loại                        |
| ROP         | Return-Oriented Programming         | Kỹ thuật khai thác lỗ hổng bằng cách kết nối các đoạn mã có sẵn trong bộ nhớ              |
| SHAP        | SHapley Additive exPlanations       | Phương pháp định lượng mức độ đóng góp của từng đặc trưng vào kết quả dự đoán             |
| SVD         | Singular Value Decomposition        | Phép phân tích ma trận thành tích các ma trận đặc trưng, dùng trong giảm chiều dữ liệu    |
| SVM         | Support Vector Machine              | Thuật toán phân loại tìm siêu phẳng tối ưu phân tách các lớp dữ liệu                      |
| TPM         | Trusted Platform Module             | Chip bảo mật phần cứng lưu trữ khóa mật mã và đảm bảo tính toàn vẹn hệ thống              |
| WDK         | Windows Driver Kit                  | Bộ công cụ và thư viện chính thức của Microsoft để phát triển driver Windows              |
| WHQL        | Windows Hardware Quality Labs       | Chương trình kiểm định và cấp chứng nhận tương thích phần cứng của Microsoft              |
| xAI         | Explainable Artificial Intelligence | Hướng nghiên cứu AI tập trung vào khả năng giải thích quá trình ra quyết định của mô hình |



# Mở đầu

## Đặt vấn đề

Trong bối cảnh chuyển đổi số quốc gia đang diễn ra mạnh mẽ, hạ tầng công nghệ thông tin (CNTT) không ngừng được mở rộng về quy mô và độ phức tạp. Hàng triệu thiết bị đầu cuối mới - từ máy trạm văn phòng, máy tính xách tay của nhân viên đến các hệ thống điều hành nghiệp vụ tại cơ quan nhà nước - được kết nối vào mạng lưới tổ chức mỗi năm. Song song với sự bùng nổ đó, ngân sách và nhân lực dành cho bảo mật thông tin thường tập trung vào lớp hạ tầng mạng, máy chủ và nền tảng đám mây, trong khi bảo mật tại tầng endpoint - tức các máy tính cá nhân và máy trạm của người dùng cuối - vẫn là mắt xích ít được chú trọng nhất trong toàn bộ chiến lược phòng thủ. Đây là một nghịch lý đáng lo ngại: chính endpoint lại là mắt xích yếu nhất (the weakest link) - nơi người dùng tương tác trực tiếp với dữ liệu, là điểm khởi phát của phần lớn các chuỗi tấn công, và đồng thời là mục tiêu tấn công phổ biến nhất của các tác nhân đe dọa trên toàn thế giới.

Thực tế đó đặt ra yêu cầu cấp bách về bảo đảm an toàn thông tin (ATTT) trên diện rộng, đặc biệt là tại các doanh nghiệp, cơ quan nhà nước và hạ tầng trọng yếu của quốc gia. Các cuộc tấn công mạng ngày càng tinh vi, có chủ đích (APT - *Advanced Persistent Threat*) và thường sử dụng các mẫu phần mềm độc hại (malware) được tùy chỉnh riêng biệt, chưa từng xuất hiện trước đó trên thế giới. Điều này khiến các hệ thống phòng thủ truyền thống, như antivirus dựa trên chữ ký (signature-based), trở nên kém hiệu quả trước các mối đe dọa mới. Theo thống kê, các hệ thống an ninh mạng phát hiện trung bình hơn 467.000 mẫu phần mềm độc hại mỗi ngày trên toàn cầu trong năm 2024, tăng mạnh so với các năm trước. [1] Tại Việt Nam, 46,15% cơ quan và doanh nghiệp bị tấn công mạng ít nhất một lần, với tổng hơn 659.000 vụ tấn công được ghi nhận, trong đó 26,14% là các cuộc tấn công có chủ đích (APT) sử dụng mã độc nằm vùng, và khoảng 14,5% bị tấn công bằng ransomware mã hoá dữ liệu. Trên thực tế, gần 75% chiến dịch APT toàn cầu có sử dụng malware, và có đến 80% trong số đó tồn tại trong hệ thống hơn 6 tháng trước khi bị phát hiện, [2] cho thấy rõ ràng các phương pháp phòng thủ truyền thống dựa trên chữ ký đang ngày càng kém hiệu quả trước các mối đe dọa tinh vi, mới xuất hiện.

Trong số các hệ điều hành phổ biến hiện nay, Windows luôn là mục tiêu bị nhắm đến nhiều nhất và liên tục nhất của các cuộc tấn công an ninh mạng. Điều này không phải ngẫu nhiên mà xuất phát trực tiếp từ bức tranh thị trường thiết bị đầu cuối (endpoint) toàn cầu. Tính đến năm 2024, Windows chiếm khoảng 72% thị phần hệ điều hành desktop trên toàn thế giới, [3] bỏ xa macOS ở mức xấp xỉ 15% và GNU/Linux ở mức chưa đến 4%. [3] Tại môi trường doanh nghiệp và cơ quan nhà nước Việt Nam, tỉ lệ này còn cao hơn do các yêu cầu tương thích phần mềm nghiệp vụ, hệ thống quản lý văn bản và phần mềm kế toán vốn được xây dựng đặc thù cho nền tảng Windows. Về phía Linux, mặc dù hệ điều hành này đang ngày càng phổ biến trên máy chủ và môi trường đám mây, thị phần desktop của nó vẫn ở mức rất nhỏ, đồng nghĩa với việc số lượng người dùng cuối bị ảnh hưởng bởi malware nhắm vào Linux là không đáng kể so với Windows. Đối với macOS, kiến trúc bảo mật sandbox tương đối chặt chẽ hơn; ngoài ra, thị phần tại khu vực Đông Nam Á - trong đó có Việt Nam - vẫn rất hạn chế, khiến nó ít trở thành mục tiêu ưu tiên của các tác nhân đe dọa. Mặt khác, thị phần lớn đồng nghĩa với lợi nhuận tấn công lớn hơn: một chiến dịch malware nhắm vào Windows có khả năng lây nhiễm và gây thiệt hại trên quy mô hàng triệu thiết bị, đây là lý do khiến Windows luôn là hệ sinh thái bị nhắm mục tiêu nhiều nhất và liên tục nhất trong lịch sử an ninh mạng. Theo báo cáo của AV-TEST, trong số các mẫu malware mới được thu thập hàng năm, hơn 90% nhắm vào nền tảng Windows [4] - một tỉ lệ phản ánh rõ mức độ ưu tiên của những kẻ tấn công đối với hệ điều hành này.

Tuy nhiên, không phải mọi loại tệp độc hại trên Windows đều mang mức độ nguy hiểm như nhau. Trong hệ sinh thái mối đe dọa trên Windows, định dạng Portable Executable (PE) - bao gồm các phần mở rộng .EXE, .DLL, .SYS, và một số dạng khác - là định dạng nhị phân thực thi gốc (native binary) của hệ điều hành, và cũng là vector tấn công nguy hiểm nhất trong số đó. Khi được khởi chạy, một tệp PE tương tác trực tiếp với nhân hệ điều hành (kernel), có thể yêu cầu và nhận bất kỳ đặc quyền nào mà người dùng hoặc hệ thống cho phép, thao túng bộ nhớ, mở kết nối mạng, ghi vào registry và tạo ra các tiến trình con - tất cả mà không cần thông qua bất kỳ lớp ứng dụng trung gian nào. Trong khi đó, các định dạng tấn công khác đều bị giới hạn bởi ít nhất một lớp phụ thuộc. Tài liệu PDF độc hại phải khai thác lỗ hổng trong trình đọc PDF (Adobe Acrobat Reader, Foxit hay các trình đọc khác) và thường bị vô hiệu hóa ngay khi ứng dụng đó được vá lỗi. Macro

trong Microsoft Office (Excel VBA, Word VBA...) yêu cầu ứng dụng Office đang chạy và người dùng phải chủ động bật tính năng macro - một rào cản ngày càng khó vượt qua hơn khi Microsoft liên tục siết chặt chính sách mặc định. Các file script như .BAT, .PS1 hay .JS cần interpreter tương ứng và thường bị kiểm soát bởi chính sách (Group Policy) hoặc các giải pháp phòng thủ lớp ứng dụng. Do đó, tệp PE thực thi là *vũ khí hoàn chỉnh và mạnh mẽ nhất* trong kho tấn công của "kẻ thù": không phụ thuộc ứng dụng trung gian, không cần người dùng bật thêm tính năng nào, chỉ cần một thao tác double-click là lập tức thực thi và tương tác với hệ thống trong phạm vi đặc quyền được cấp.

Hơn nữa, tệp PE còn đóng vai trò là *payload cuối cùng* (final payload) trong phần lớn các chuỗi tấn công đa giai đoạn. Dù kẻ tấn công bắt đầu bằng một email lừa đảo (phishing), một tài liệu Office có macro hay một tệp PDF được nhúng shellcode, mục tiêu cuối cùng của chúng gần như luôn là thả xuống (drop) và thực thi một tệp PE độc hại trên máy nạn nhân. Ransomware, Remote Access Trojan (RAT), rootkit, keylogger, spyware - tất cả các lớp malware nguy hiểm nhất đều được triển khai dưới dạng tệp PE. Theo dữ liệu từ bộ dữ liệu EMBER - một trong những bộ dữ liệu malware PE lớn nhất hiện có - có hơn một triệu mẫu PE độc hại được thu thập và phân tích, [5] phản ánh sự phổ biến và tầm quan trọng của định dạng này trong nghiên cứu phát hiện mã độc. Ngoài ra, định dạng PE với cấu trúc header phong phú - bao gồm DOS header, PE header, các section header, bảng import/export, bảng tài nguyên và nhiều trường siêu dữ liệu khác - cung cấp một không gian đặc trưng (feature space) dồi dào cho các mô hình học máy khai thác, điều mà các định dạng tấn công khác như script hay shellcode thô không có được.

Do đó, việc nghiên cứu và triển khai một mô-đun phát hiện mã độc định dạng Windows Portable Executable (PE) dựa trên học máy (machine learning) là yêu cầu cấp thiết, đồng thời mang ý nghĩa khoa học và thực tiễn rõ rệt. Khác với các phương pháp truyền thống dựa trên chữ ký, các mô hình học máy có khả năng tự động học đặc trưng từ dữ liệu, cho phép phân tích sâu các đặc điểm tĩnh (cấu trúc tệp, PE header, bảng import/export, entropy, chuỗi ký tự...) cũng như hành vi của mã độc. Nhờ đó, hệ thống có thể phát hiện hiệu quả các mẫu mã độc mới, chưa từng xuất hiện trước đây (zero-day malware), vốn là điểm yếu lớn của các giải pháp antivirus truyền thống. Việc áp dụng học máy trong bài toán phát hiện

mã độc không chỉ giúp nâng cao tỷ lệ phát hiện (detection rate) mà còn góp phần giảm thiểu tỷ lệ cảnh báo sai (false positive), từ đó nâng cao độ tin cậy của các hệ thống phòng thủ an toàn thông tin. Đặc biệt, trong bối cảnh các chiến dịch tấn công có chủ đích (APT) ngày càng phổ biến và thường sử dụng mã độc được tùy chỉnh riêng cho từng mục tiêu, khả năng phát hiện dựa trên hành vi và đặc trưng tổng quát của các mô hình học máy đóng vai trò quan trọng trong việc tăng cường chiều sâu phòng thủ (defense in depth) cho các hệ thống CNTT.

Bên cạnh ý nghĩa về mặt kỹ thuật, đề tài còn có giá trị nghiên cứu và đào tạo rõ rệt. Việc xây dựng mô-đun phát hiện mã độc PE dựa trên học máy giúp người nghiên cứu tiếp cận một cách hệ thống các kiến thức liên ngành, bao gồm kiến trúc hệ điều hành Windows, định dạng tệp PE, kỹ thuật phân tích mã độc, xử lý dữ liệu, cũng như các thuật toán học máy hiện đại. Qua đó, đề tài góp phần củng cố nền tảng lý thuyết và kỹ năng thực hành trong lĩnh vực an toàn thông tin và trí tuệ nhân tạo - vốn là những lĩnh vực then chốt trong khoa học máy tính hiện nay. Đề tài cũng góp phần nâng cao năng lực nghiên cứu và tự chủ công nghệ trong lĩnh vực an ninh mạng, thông qua việc chủ động xây dựng và đánh giá các mô hình phát hiện mã độc, thay vì hoàn toàn phụ thuộc vào các giải pháp đóng và công nghệ nước ngoài, đồng thời đáp ứng yêu cầu bảo đảm an toàn thông tin trong quá trình chuyển đổi số tại Việt Nam, phù hợp với các định hướng và chiến lược phát triển an toàn, an ninh mạng do Chính phủ đề ra. [6]

Với những lý do trên, đề tài “*Ứng dụng học máy trong phát hiện mã độc PE trên Windows*” là một hướng nghiên cứu có giá trị khoa học, tính thời sự và khả năng ứng dụng thực tiễn cao, đồng thời phù hợp với mục tiêu đào tạo và nghiên cứu của các chuyên ngành liên quan đến an ninh, an toàn thông tin trong các cơ sở giáo dục đại học hiện nay.

## **Mô tả bài toán**

Khóa luận tập trung giải quyết hai bài toán có liên hệ mật thiết với nhau: *bài toán nghiên cứu* phương pháp học máy để phân loại file PE (phân loại nhị phân, với nhãn phân loại là lành tính hoặc mã độc), và *bài toán công nghệ* về triển khai dịch vụ phát hiện mã độc trên máy endpoint (Windows).

Cụ thể, bài toán nghiên cứu mà khóa luận giải quyết:

- **Đầu vào:** Một file thực thi định dạng PE trên Windows, cùng với nhãn phân loại tương ứng (trong giai đoạn huấn luyện).
- **Đầu ra:** Nhãn phân loại (lành tính - benign hoặc mã độc - malware).
- **Phạm vi:** Phân tích tĩnh (static analysis), không thực thi file trong môi trường sandbox. Dữ liệu thực nghiệm sử dụng tập EMBER2024.
- **Giả định:** File PE hợp lệ về mặt cấu trúc và có thể đọc được bằng các công cụ phân tích tĩnh thông thường.
- **Phương pháp:** Xây dựng và so sánh các pipeline học máy kết hợp đặc trưng từ EMBER2024 với đặc trưng năng lực mã độc được trích xuất bằng công cụ Capa. Áp dụng kỹ thuật giảm chiều TruncatedSVD để nén đặc trưng Capa, và SHAP để giải thích kết quả dự đoán của mô hình.

Bài toán công nghệ mà khóa luận giải quyết:

- **Đầu vào:** Một file PE cần kiểm tra, được cung cấp thông qua giao diện người dùng (tải lên thủ công) hoặc được phát hiện tự động bởi driver khi có tiến trình thực thi trên máy endpoint.
- **Đầu ra:** Kết quả phân loại (lành tính hoặc mã độc), danh sách các năng lực/hành vi đặc trưng được trích xuất, giải thích trực quan quyết định của mô hình với kỹ thuật xAI, và lịch sử các lần quét được lưu trữ để tra cứu.
- **Phạm vi:** Xây dựng hệ thống gồm nhiều thành phần chạy trên máy endpoint Windows, bao gồm kernel driver (KMDF), Windows service, module trích xuất đặc trưng (C++), giao diện người dùng trên máy endpoint, và dịch vụ phân tích chuyên sâu online (deep-analysis-server). Phần triển khai trong môi trường thực tế (production) được tiếp cận ở mức mô phỏng và phân tích kiến trúc; đề tài không đặt mục tiêu triển khai hoàn chỉnh trên hệ thống quy mô lớn.
- **Giả định:** Môi trường thực thi là máy tính cá nhân hoặc máy trạm chạy Windows, có đủ quyền để cài đặt driver và Windows service. Riêng dịch vụ phân tích file PE chuyên sâu online được triển khai dưới dạng ứng dụng web trên hệ điều hành Linux.

- **Phương pháp:** Tích hợp pipeline học máy từ bài toán nghiên cứu vào kiến trúc dịch vụ gồm hai chế độ - antivirus offline chạy nền trên endpoint và dịch vụ phân tích sâu online - kết hợp các kỹ thuật lập trình hệ thống Windows như lập trình driver (KMDF), Windows service và Windows API.

## Mục tiêu và phạm vi nghiên cứu của khóa luận

Khóa luận đặt ra ba mục tiêu chính như sau:

1. **Nghiên cứu kỹ thuật trích xuất năng lực mã độc (capability extraction):** Tìm hiểu và làm chủ các phương pháp phân tích tĩnh file PE nhằm trích xuất các năng lực và hành vi đặc trưng của mã độc. Đây là nền tảng để xây dựng bộ đặc trưng (feature set) có ý nghĩa ngữ nghĩa cao hơn so với các đặc trưng cấu trúc thông thường, từ đó nâng cao hiệu quả phân loại của mô hình học máy.
2. **Đánh giá thực nghiệm trên tập dữ liệu thực tế:** Huấn luyện và đánh giá các mô hình học máy trên tập dữ liệu EMBER2024 - một trong những tập dữ liệu benchmark chuẩn được công nhận, có quy mô lớn phù hợp cho bài toán nghiên cứu, xây dựng mô hình phát hiện mã độc PE. Thực nghiệm nhằm làm rõ bộ đặc trưng nào mang lại hiệu quả phân loại tốt nhất, cũng như đóng góp của kỹ thuật capability extraction vào hiệu quả tổng thể của mô hình học máy.
3. **Thiết kế và mô phỏng kiến trúc agent bảo mật trên endpoint:** Nghiên cứu và thiết kế kiến trúc triển khai mô hình phát hiện mã độc dưới dạng một agent chạy trực tiếp trên máy endpoint Windows. Phần này tiếp cận ở mức mô phỏng và phân tích kiến trúc, tập trung vào việc tìm hiểu và áp dụng các kỹ thuật lập trình hệ thống Windows bao gồm lập trình driver (KMDF), lập trình Windows service và sử dụng Windows API. Đề tài không đặt mục tiêu triển khai hoàn chỉnh trên hệ thống quy mô lớn.

Do đó, phạm vi nghiên cứu của khóa luận có thể được tóm tắt như sau:

- **Đối tượng phân tích:** File thực thi định dạng PE (.exe, .dll) trên hệ điều hành Windows.

- **Phương pháp phân tích:** Phân tích tĩnh (static analysis); không bao gồm phân tích động (dynamic analysis) hay phân tích trong sandbox.
- **Dữ liệu:** Tập dữ liệu EMBER2024 dùng cho huấn luyện và đánh giá mô hình.
- **Mô hình học máy:** Tập trung vào các mô hình học máy truyền thống (gradient boosting).
- **Triển khai:** Ở mức thiết kế kiến trúc và mô phỏng thành phần (Proof-of-Concept), phù hợp mục tiêu nghiên cứu, áp dụng các kỹ thuật lập trình hệ thống trên hệ điều hành Windows.

## Cấu trúc khóa luận

Phần còn lại của khóa luận được tổ chức thành các chương như sau:

**Chương 1 - Cơ sở lý thuyết và công nghệ sử dụng:** Trình bày các kiến thức nền tảng cần thiết để hiểu các phương pháp được đề xuất ở các chương sau, bao gồm: tổng quan về mã độc và định dạng file PE, các phương pháp phát hiện mã độc, kỹ thuật trích xuất năng lực mã độc (capability extraction) bằng công cụ Capa, các mô hình học máy và kỹ thuật giảm chiều dữ liệu được sử dụng, phương pháp giải thích mô hình (xAI) với SHAP, các công trình nghiên cứu liên quan, và các công nghệ, thư viện áp dụng trong đề tài.

**Chương 2 - Thiết kế và đánh giá các pipeline học máy trong phát hiện mã độc:** Trình bày kiến trúc tổng thể của giải pháp, thiết kế chi tiết ba pipeline học máy được đề xuất dựa trên các tổ hợp đặc trưng khác nhau (EMBER2024 thuần, EMBER2024 kết hợp Capa one-hot, EMBER2024 kết hợp Capa TruncatedSVD), và kết quả so sánh thực nghiệm giữa các pipeline trên tập dữ liệu EMBER2024. Từ đó xác định pipeline phù hợp nhất để đưa vào triển khai ở Chương 3.

**Chương 3 - Phát triển dịch vụ phát hiện mã độc trên máy endpoint và dịch vụ phân tích mã độc online:** Trình bày phân tích yêu cầu và thiết kế chi tiết hệ thống agent chạy trên máy endpoint Windows, bao gồm các thành phần: kernel driver (KMDF), Windows service, module trích xuất đặc trưng, server/dịch vụ phân tích chuyên sâu online tích hợp Capa và SHAP, và giao diện người dùng trên máy endpoint. Chương này cũng trình bày kết quả cài đặt, các kịch bản kiểm thử và đánh giá hệ thống.

**Kết luận và hướng phát triển:** Tổng kết các kết quả đạt được của cả hai bài toán nghiên cứu và công nghệ, nêu rõ những hạn chế còn tồn tại, và đề xuất hướng phát triển trong tương lai.



# Chương 1

## Cơ sở lý thuyết

### 1.1 Tổng quan về mã độc PE trên hệ điều hành Windows

#### 1.1.1 Tổng quan về mã độc (malware)

Mã độc (malware - malicious software) là thuật ngữ dùng để chỉ các chương trình phần mềm được thiết kế với mục đích gây hại cho hệ thống máy tính, đánh cắp dữ liệu, gián điệp người dùng, hoặc thực hiện các hành vi trái phép khác mà không có sự cho phép của chủ sở hữu hệ thống. Mã độc có thể lây lan qua nhiều con đường khác nhau như email, phần mềm giả mạo, thiết bị lưu trữ di động hoặc qua mạng Internet.

Các nhà nghiên cứu chia mã độc thành nhiều loại khác nhau, và mỗi loại mã độc lại có thể được phân loại chi tiết hơn (chia thành các phân nhóm nhỏ hơn). Một số loại mã độc phổ biến trong hệ thống máy tính và hệ thống mạng bao gồm: [7]

1. **Virus máy tính (Computer virus):** Lây nhiễm bằng cách chèn mã độc vào các tập tin hợp lệ (vật chủ) và phát tán khi người dùng mở các tập tin đó.
2. **Sâu (Worm):** Loại mã độc có khả năng tự nhân bản và lây lan nhanh qua mạng mà không cần đến tập tin chủ (vật chủ) như virus.
3. **Trojan:** Loại mã độc nguy trang dưới dạng phần mềm hợp lệ để đánh lừa người dùng cài đặt.
4. **Mã độc tống tiền (Ransomware):** Mã hóa dữ liệu của nạn nhân và đòi tiền chuộc để khôi phục.
5. **Phần mềm gián điệp (Spyware):** Theo dõi và ghi lại hành vi người dùng để đánh cắp thông tin nhạy cảm. Một trong những dạng spyware đầu tiên là keylogger - ghi lại các thao tác gõ phím của người dùng. Một số phần mềm gián điệp hiện đại còn khai thác thông tin kênh bên (side-channel) từ các đặc tính vật lý của hệ thống (như điện năng tiêu thụ, nhiệt độ, thời gian thực

thi... ) nhằm phục vụ cho việc phân tích, giải mã thông tin mã hóa, bẻ khóa hệ thống, truy cập trái phép...

6. **Rootkit:** Loại mã độc ẩn mình sâu trong hệ thống, che giấu các tiến trình hoặc tệp tin liên quan để tránh bị phát hiện. Rootkit có thể làm được điều này là nhờ nó can thiệp sâu vào kernel hoặc các thành phần hệ thống quan trọng, cho phép nó kiểm soát hệ thống cũng như các phần mềm bảo mật.

Mã độc có thể được chứa trong nhiều loại tệp tin, thậm chí cả PDF. Trên hệ điều hành Windows, một trong những loại tệp tin chứa mã độc phổ biến hơn cả là định dạng PE (Portable Executable) - định dạng tệp tin thực thi tiêu chuẩn của Windows, thường có phần mở rộng .exe, .dll, .sys... Mã độc PE lợi dụng cấu trúc đặc trưng của định dạng này, cũng như những lỗ hổng trong cách thức hệ điều hành Windows nạp (load) chúng vào bộ nhớ, từ đó chèn mã độc vào các vùng nhớ, giấu mã hoặc thực hiện các hành vi tinh vi như tải xuống mã độc bổ sung, thực thi theo điều kiện, hoặc chống phân tích ngược...

Mã độc hiện đại thường là một phần trong các chiến dịch tấn công có chủ đích (APT - Advanced Persistent Threat). Các chiến dịch này được tổ chức bài bản, kéo dài trong thời gian dài và thường nhắm vào các tổ chức, doanh nghiệp hoặc cơ quan chính phủ để đánh cắp thông tin, gây rối hoạt động hoặc phá hoại. [8]

Hiện nay, thế giới đang chứng kiến sự gia tăng mạnh mẽ cả về số lượng lẫn mức độ tinh vi của các loại mã độc. Theo báo cáo của AV-TEST (2024), [4] trung bình mỗi ngày có hơn 450.000 mẫu mã độc mới được phát hiện trên toàn cầu, cho thấy mức độ phát triển nhanh chóng và khó kiểm soát của các mối đe dọa này. Đặc biệt, mã độc dạng PE (Portable Executable) vẫn chiếm tỷ lệ lớn trong các cuộc tấn công nhắm vào hệ điều hành Windows, nền tảng phổ biến trong cả môi trường cá nhân lẫn doanh nghiệp. Do khả năng thực thi dễ dàng, linh hoạt và tích hợp sâu vào hệ thống, mã độc PE có thể lợi dụng nhiều kỹ thuật để tránh bị phát hiện, đồng thời phát tán nhanh chóng trong mạng nội bộ của tổ chức. Điều này đặt ra rủi ro nghiêm trọng về mất mát dữ liệu, gián đoạn hoạt động kinh doanh và thậm chí là ảnh hưởng đến an ninh quốc gia nếu các cơ quan chính phủ bị xâm nhập.

Trong bối cảnh chuyển đổi số đang diễn ra mạnh mẽ tại Việt Nam cũng như nhiều quốc gia trên thế giới theo định hướng Cách mạng công nghiệp 4.0, các hệ thống thông tin ngày càng trở nên phức tạp, kết nối chặt chẽ và phụ thuộc nhiều hơn vào

công nghệ. Điều này đồng nghĩa với việc mở rộng bề mặt tấn công (attack surface) cho các đối tượng xấu và làm gia tăng nguy cơ bị mã độc tấn công, đặc biệt trong các tổ chức trọng yếu như ngân hàng, y tế, cơ quan nhà nước hay doanh nghiệp hạ tầng. Chính vì vậy, nhu cầu xây dựng và triển khai các phương pháp phát hiện và ngăn chặn mã độc hiệu quả, trong đó có mã độc PE, đang trở thành một vấn đề cấp bách, không chỉ đối với các tổ chức chuyên về an ninh mạng mà còn đối với toàn bộ hệ sinh thái số quốc gia.

### 1.1.2 Tổng quan về các phương pháp phát hiện và ngăn chặn mã độc

Mã độc ngày nay không chỉ đơn giản là những đoạn mã phá hoại mà đã phát triển thành các công cụ tấn công tinh vi, được sử dụng trong các chiến dịch APT, ransomware, gián điệp công nghiệp và thậm chí là tấn công vào cơ sở hạ tầng quốc gia. Trước thực trạng đó, không một giải pháp đơn lẻ nào có thể đảm bảo an toàn tuyệt đối, mà cần đến một chiến lược an ninh mạng nhiều lớp (multi-layered security), kết hợp các phương pháp phát hiện và ngăn chặn từ nhiều góc độ, chẳng hạn như:

- **Lớp bảo vệ mạng:** Phát hiện mã độc qua lưu lượng mạng, sandbox, IDS/IPS.
- **Lớp cổng kết nối:** Ngăn chặn mã độc qua email, web, tệp tải về.
- **Lớp endpoint:** Phát hiện và phản ứng trực tiếp trên thiết bị đầu cuối - nơi mã độc thực sự được thực thi.
- **Lớp ứng dụng/cloud:** Bảo vệ dịch vụ, dữ liệu và nền tảng trong môi trường đám mây.
- **Lớp phân tích và phản ứng:** Bao gồm Threat Intelligence, SOC, SIEM, Threat Hunting...

Trong đó, việc bảo vệ endpoint (thiết bị đầu cuối) [9] đóng vai trò đặc biệt quan trọng bởi đây là nơi mã độc thực sự tác động đến người dùng, dữ liệu và tài sản số của tổ chức. Các thiết bị đầu cuối như máy tính cá nhân, máy chủ, laptop nhân viên, thiết bị IoT... là đích đến cuối cùng của phần lớn mã độc. Dù tấn công bắt đầu từ email, trình duyệt hay thiết bị USB, mã độc vẫn cần được thực thi trên endpoint để gây hại. Do đó, bảo vệ endpoint là tuyến phòng thủ cuối cùng và quan

trọng nhất, giúp tổ chức phát hiện sớm các hành vi đáng ngờ ngay tại thiết bị, ngăn chặn mã độc trước khi nó thực hiện hành vi phá hoại hoặc lây lan, cũng như có khả năng phản ứng kịp thời, cách ly thiết bị, thu thập bằng chứng phục vụ điều tra. Một giải pháp phát hiện mã độc ngay tại máy endpoint của người dùng có thể được coi là rất hữu ích và hiệu quả.

Trên cơ sở đó, việc khóa luận tập trung phát triển tính năng agent phát hiện mã độc chạy trên máy endpoint là hoàn toàn phù hợp với nhu cầu thực tiễn. Thay vì chỉ dừng lại ở việc xây dựng mô hình học máy thuần túy, khóa luận hướng đến tích hợp mô hình vào một hệ thống có thể vận hành thực sự trên môi trường Windows - nơi mã độc PE thực sự hoạt động và gây hại. Tuy nhiên, việc xây dựng một giải pháp bảo vệ endpoint hoàn chỉnh đòi hỏi sự kết hợp của nhiều lớp kỹ thuật khác nhau, từ phân tích và phân loại file cho đến giám sát hành vi thời gian thực ở tầng hệ điều hành. Để giải quyết hiệu quả bài toán phân loại, cần có một pipeline học máy đủ mạnh, được xây dựng trên bộ đặc trưng phong phú và có ý nghĩa ngữ nghĩa. Bên cạnh đặc trưng cấu trúc thông thường của file PE, việc bổ sung thông tin về *năng lực* (capability) mà file có thể thực hiện - chẳng hạn khả năng mã hóa dữ liệu, kết nối mạng, leo thang đặc quyền hay can thiệp vào tiến trình hệ thống - mang lại góc nhìn ngữ nghĩa sâu hơn, giúp mô hình phân biệt mã độc hiệu quả hơn so với chỉ dựa vào đặc trưng thống kê đơn thuần. Chi tiết việc xây dựng và tích hợp pipeline sẽ được trình bày, làm rõ trong Chương 2 và Chương 3.

### 1.1.3 Tổng quan về x86/amd64 Assembly

Trong lĩnh vực an toàn thông tin, đặc biệt là trong phân tích và phát hiện mã độc trên hệ điều hành Windows, kiến thức nền tảng về kiến trúc vi xử lý x86/amd64 và định dạng tệp thực thi PE (Portable Executable) đóng vai trò rất quan trọng. Đây là hai yếu tố kỹ thuật cốt lõi giúp hiểu rõ cách mã độc hoạt động, cách chúng lẩn tránh các cơ chế bảo vệ, cũng như cách các công cụ phân tích có thể phát hiện và vô hiệu hóa chúng.

Windows hiện vẫn là hệ điều hành phổ biến nhất cho người dùng cá nhân và doanh nghiệp. Các mã độc nhắm vào nền tảng này - đặc biệt là dạng mã độc PE - tiếp tục chiếm tỷ lệ lớn trong các chiến dịch tấn công thực tế. Với khả năng can thiệp trực tiếp vào bộ nhớ, ẩn mình trong tiến trình hợp pháp, hoặc khai thác các kỹ thuật cấp thấp, mã độc PE đòi hỏi người phòng thủ phải nắm vững kiến thức kỹ

thuật ở tầng thấp (low-level), cụ thể là:

- Cấu trúc và cách thức hoạt động của mã máy (Assembly) trên nền kiến trúc x86/amd64. Hiện nay, đây vẫn là kiến trúc tập lệnh (ISA) phổ biến nhất được sử dụng bởi các thiết bị Windows cũng như các tệp PE.
- Định dạng và đặc điểm nội tại của các tệp PE.

Việc hiểu được cách các mã máy được sinh ra, tổ chức trong một tệp PE, và cuối cùng được nạp vào bộ nhớ và thực thi, là nền tảng không thể thiếu cho các chuyên gia làm việc trong các lĩnh vực như phân tích mã độc (malware analysis), kỹ thuật dịch ngược (reverse engineering) và xây dựng hệ thống phát hiện mã độc bằng học máy (ML-based malware detection). Khóa luận này sẽ trình bày chi tiết về hai chủ đề chính: kiến trúc x86/amd64 và định dạng tệp PE, trong đó đặc biệt nhấn mạnh tới các điểm dễ bị lợi dụng bởi mã độc cũng như các đặc trưng hữu ích cho việc phát hiện và phân tích chúng.

Hai kiến trúc x86 (32-bit) và amd64 (64-bit, còn gọi là x86-64) là nền tảng xử lý phổ biến nhất trên máy tính cá nhân và máy chủ sử dụng hệ điều hành Windows. Cả hai kiến trúc này đều thuộc họ CISC (Complex Instruction Set Computing), với tập lệnh đa dạng và phong phú, cho phép lập trình viên thao tác trực tiếp với bộ nhớ, thanh ghi, và luồng điều khiển chương trình. [10] [11] [12]

**Về thanh ghi và cấu trúc bộ xử lý**, kiến trúc x86 cung cấp 8 thanh ghi 32-bit tổng quát: EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP. Kiến trúc amd64 mở rộng số lượng thanh ghi tổng quát lên 16, với kích thước 64-bit: RAX, RBX, RCX, RDX, RSI, RDI, RBP, RSP và 8 thanh ghi còn lại R8-R15. Các thanh ghi ESP, RSP (Stack Pointer) và EIP, RIP (Instruction Pointer) đóng vai trò quan trọng trong điều khiển luồng thực thi, và cũng bị lợi dụng bởi một số kỹ thuật tấn công phổ biến, chẳng hạn return-oriented programming (ROP).

**Về chế độ vận hành**, hai kiến trúc đều cung cấp các chế độ real mode, protected mode và long mode. Trong real mode (chế độ thực), CPU không thực hiện bảo vệ bộ nhớ cấp phần cứng (hardware-level memory protection) - tức không gian địa chỉ luôn ánh xạ 1:1 với địa chỉ thực của bộ nhớ. Trong khi đó, protected mode (chế độ bảo vệ) được sử dụng rộng rãi trong x86, hỗ trợ phân vùng bộ nhớ và quyền truy cập, không cho phép các chương trình đọc, ghi trực tiếp vào địa chỉ thực của

bộ nhớ mà phải sử dụng địa chỉ ảo (virtual addresses). Long mode là chế độ chỉ có ở amd64, cho phép truy cập không gian bộ nhớ lên tới 64 bit, sử dụng paging để bảo vệ vùng nhớ.

**Về tập lệnh và cú pháp**, hai kiến trúc đều có tập lệnh hợp ngữ (assembly) rất đa dạng và có nhiều lệnh (mnemonics) tương đồng, bao gồm thao tác số học (ADD, SUB, INC, DEC), logic (AND, OR, XOR), đọc, ghi dữ liệu (MOV, PUSH, POP), điều khiển luồng (CALL, JMP, RET, LOOP, JZ, JNZ...). Cú pháp phổ biến của hợp ngữ trên Windows là Intel syntax. Ngoài ra còn có AT&T syntax. Cần lưu ý rằng một đoạn mã máy có thể được dịch trở lại thành mã hợp ngữ khác nhau do sự khác biệt về syntax hay các biến thể của các trình hợp dịch (assembler) và dịch ngược (disassembler).

Ngoài ra, trong phân tích mã độc và nghiên cứu file thực thi nhị phân (binary), việc hiểu về quy ước hay giao ước gọi hàm (calling convention) cũng rất quan trọng. Một calling convention quyết định cách các tham số được truyền vào hàm, cách hàm đó lưu giá trị trả về, và cách vùng nhớ stack được quản lý khi gọi hàm đó. Các calling convention phổ biến được cho trong 1.1. Biết được calling convention, người ta có thể xác định điểm vào hàm, tham số, và luồng điều khiển khi xem disassembly - điều cực kỳ quan trọng khi phân tích thủ công các tệp thực thi PE. Hơn nữa, mã độc thường thao túng stack hoặc gọi các API hệ thống một cách gián tiếp (indirect call, ví dụ dùng `LoadLibrary` và `GetProcAddress` trên Windows), khiến việc hiểu rõ calling convention là cần thiết để truy vết hành vi thực sự của mã độc. Một số mã độc còn cố tình vi phạm calling convention chuẩn để gây lỗi, gây khó hiểu (confuse) cho các công cụ phân tích tĩnh như disassembler, công cụ dịch ngược sang ngôn ngữ C (chẳng hạn Ghidra, IDA)...

Có thể thấy, các lệnh hợp ngữ chỉ thực hiện các thao tác đơn giản, hơn nữa các calling convention chỉ là giao ước giữa các hàm với nhau, hoàn toàn không bị bắt buộc hay bị kiểm tra bởi CPU. Do đó, kiến trúc x86/amd64 tạo điều kiện cho các cuộc tấn công khai thác lỗ hổng ở tầng thấp, chẳng hạn buffer overflow, return-oriented programming (ROP) hay shellcode injection. Nhiều mã độc được viết bằng assembly tối giản, nhỏ gọn, khó phát hiện khi chưa giải mã hoặc unpack. Việc hiểu và áp dụng hợp ngữ trên Windows, đặc biệt là x86 và amd64, đóng vai trò quan trọng trong phân tích mã độc, cũng như nghiên cứu giải pháp phát hiện và ngăn chặn mã độc mới.

### 1.1.4 Tổng quan về định dạng PE (Portable Executable) trên Windows

Định dạng PE (Portable Executable) [13] [14] là định dạng chuẩn cho các tệp thực thi trên hệ điều hành Windows, bao gồm `.exe`, `.dll`, `.sys`, và một số định dạng khác. PE là phần mở rộng của định dạng COFF (Common Object File Format), được Microsoft phát triển để hỗ trợ nạp mã máy vào bộ nhớ và thực thi một cách linh hoạt. Mỗi tệp PE là một tổ chức chặt chẽ giữa mã máy (executable code), dữ liệu (data), siêu dữ liệu (metadata), và thông tin cần thiết để hệ điều hành có thể thực thi chương trình đúng ngữ cảnh.

Cấu trúc tổng quát của tệp PE gồm 5 bộ phận chính kế tiếp nhau:

- **DOS Header** (còn gọi là MS-DOS Header): vùng có kích thước 64 bytes nhằm mục đích tương thích ngược với hệ điều hành MS-DOS. Nếu chạy tệp PE trong DOS, header này khiến DOS nhận diện tệp là một tệp thực thi bình thường và sẽ thực thi phần mã máy trong DOS stub. Đối với Windows NT, bộ phận này chỉ còn tồn tại vì lý do lịch sử. Đặc trưng của DOS Header là mở đầu bằng 2 byte "MZ" (0x5A4D) - được gọi là *magic number* - là dấu hiệu nhận diện tệp thực thi của DOS.
- **DOS Stub**: vùng chứa mã máy chạy được trong DOS. Thông thường vùng này in ra dòng thông báo *"This program cannot be run in DOS mode"* rồi thoát chương trình. Phần này chỉ tồn tại vì lý do tương thích ngược; hệ điều hành Windows không thực thi mã máy trong phần này.
- **PE Header** (NT Header): vùng chứa các metadata - thông tin tổng thể về tệp tin, chẳng hạn kiến trúc CPU của mã thực thi, số sections, và characteristics (đặc tính tệp tin: executable, system file...). Đặc trưng của vùng này là mở đầu bằng một signature chứa 4 bytes 0x50450000, tức "PE\0\0" theo ASCII.
- **Section Headers**: chứa thông tin từng section, bao gồm địa chỉ bắt đầu, kích thước và các thuộc tính của section.
- **Sections**: vùng chứa nội dung thực sự của các section, với vị trí và thuộc tính đã được định nghĩa trong Section Headers. Các sections điển hình bao gồm:
  - `.text`: chứa mã thực thi.
  - `.data`: biến toàn cục/biến tĩnh có thể đọc và ghi.

- **.rdata**: dữ liệu chỉ đọc.
- **.reloc**: bảng ánh xạ lại địa chỉ (relocations).
- **.rsrc**: tài nguyên (resources) như ảnh, icon, form...

Kiến trúc x86/amd64 và định dạng PE trên Windows, tuy linh hoạt và hiệu quả cho thực thi mã, nhưng cũng ẩn chứa nhiều điểm yếu có thể bị mã độc lợi dụng để xâm nhập, ẩn mình và hoạt động âm thầm trong hệ thống. Dưới đây liệt kê một số điểm yếu tiêu biểu:

- **Bộ nhớ phẳng (flat) và chia sẻ (shared)**: Trên Windows, tiến trình có thể truy cập toàn bộ không gian địa chỉ ảo của mình. Nếu không có cơ chế bảo vệ như DEP (Data Execution Prevention) hay ASLR (Address Space Layout Randomization), mã độc có thể thay đổi dữ liệu, ghi đè mã, hoặc thực thi từ vùng dữ liệu một cách dễ dàng.
- **Thiếu kiểm soát chặt chẽ giữa code và data**: Kết hợp với kỹ thuật code injection (tiêm mã vào tiến trình khác), mã độc có thể viết và thực thi mã ở bất kỳ đâu nếu có quyền phù hợp. Chẳng hạn, mã độc có thể che giấu payload bằng cách mã hóa, sau đó giải mã ra một vùng nhớ và đặt quyền thực thi trên vùng đó để kích hoạt mã độc.
- **Indirect calls/jumps**: Assembly cho phép gọi hàm hoặc nhảy đến địa chỉ được lưu trong thanh ghi hoặc bộ nhớ (ví dụ: `CALL [EAX]`, `JMP RAX`). Mã độc dùng kỹ thuật này để ẩn đích thực thi, gây khó khăn cho phân tích tĩnh và các công cụ dịch ngược.
- **Gọi API động**: Mã độc thường sử dụng `LoadLibrary`, `GetProcAddress`, resolve hashing, hoặc kỹ thuật gọi hàm theo địa chỉ offset để lẩn tránh các bộ lọc theo chuỗi API tĩnh.

Lợi dụng linh hoạt các điểm yếu trên, người ta đã phát triển rất nhiều kỹ thuật khai thác nhắm vào tập tin PE và môi trường thực thi Windows, chẳng hạn:

- **Section spoofing**: Tạo các section giả với tên hợp lệ như `.text` hoặc `.rdata` nhưng chứa mã độc, gây nhầm lẫn cho công cụ phân tích; hoặc tạo section tùy ý chứa mã độc (`.xyz`, `.UPX`, `.bss`,...).



- **PE header corruption:** Thay đổi hoặc làm hỏng PE Header nhằm làm gián đoạn công cụ phân tích hoặc antivirus.
- **Entry point manipulation:** Thay đổi Entry Point để trỏ vào mã độc thay vì hàm `main` hoặc `WinMain` thông thường.
- **Import table spoofing:** Tạo bảng IAT (Import Address Table) giả, hoặc mã hóa các chuỗi API để vượt qua signature detection.
- **Packing/obfuscation:** Sử dụng packer, crypter để mã hóa toàn bộ phần `.text`, chỉ giải mã khi chạy, từ đó vô hiệu hóa phân tích tĩnh thông thường. Một số kỹ thuật làm nhiễu trong Relocation và Exception Table cũng thường được sử dụng.
- **Process hollowing/doppelganging:** Tạo một tiến trình hợp pháp (như `notepad.exe`), rồi ghi đè bộ nhớ của tiến trình đó bằng mã độc, giúp chạy dưới tên process hợp lệ.
- **Reflective DLL injection:** Tiêm DLL trực tiếp vào bộ nhớ mà không cần ghi xuống đĩa, tránh bị antivirus phát hiện.
- **Syscall direct invocation:** Mã độc bỏ qua các API cấp cao và trực tiếp gọi syscall của Windows, vượt qua hook của EDR (Endpoint Detection and Response).
- **Return-oriented programming (ROP):** Dùng các đoạn mã ngắn (gadgets) có sẵn trong thư viện để tạo luồng thực thi, giúp né DEP và gây khó khăn cho phân tích dòng điều khiển.

Việc hiểu rõ đặc điểm và điểm yếu của kiến trúc x86/amd64 và định dạng PE trên Windows là điều kiện tiên quyết để xây dựng các hệ thống phát hiện mã độc hiệu quả. Đây cũng là nền tảng để áp dụng các kỹ thuật như disassembly, symbolic execution, feature extraction và học máy trong phân tích mã độc hiện đại.

### 1.1.5 Tổng quan về các phương pháp phân tích mã độc PE

Trước khi đưa vào mô hình học máy để phát hiện và phân loại (là lành tính hay có chứa mã độc, mã độc thuộc loại nào...), một tập tin PE cần được phân tích và

trích xuất đặc trưng. Có ba phương pháp phân tích tập tin PE cơ bản: phân tích tĩnh, phân tích động và phương pháp lai. [15, 16]

**Phân tích tĩnh** (static analysis) trích xuất đặc trưng mà không cần thực thi mẫu mã độc. Nhiều loại đặc trưng tĩnh khác nhau có thể được thu thập, bao gồm: giá trị băm (hash), chuỗi N-gram, mã vận hành (opcode), chuỗi văn bản (string), và thông tin tiêu đề PE (PE header). Dựa trên những đặc trưng này, các phần mềm phát hiện mã độc như chương trình chống virus, hệ thống phát hiện xâm nhập (IDS), v.v. được xây dựng. Để kiểm tra mẫu mã độc ở cấp độ mã máy, các tập tin thực thi được dịch ngược thành mã hợp ngữ (assembly) thông qua các công cụ như OllyDbg, IDA Pro, Capstone và WinDbg. Mã hợp ngữ sau đó được kiểm tra nhằm xác định luồng thực thi, cấu trúc hoặc mẫu hành vi độc hại, từ đó phục vụ phát hiện các biến thể mới của mã độc hiện có. Tuy nhiên, phân tích mã hợp ngữ là quá trình tốn thời gian. Hơn nữa, các kỹ thuật làm rối mã (code obfuscation) - như mã hóa đoạn mã, hoán đổi thứ tự lệnh, hay chèn mã không thực thi (dead code) - còn khiến việc phân tích trở nên phức tạp hơn.

**Phân tích động** (dynamic analysis) thực thi tập tin PE và ghi lại các hoạt động của chương trình trong thời gian chạy, bao gồm: thao tác với hệ thống tập tin, thay đổi khóa registry, tạo tiến trình khác, các hoạt động mạng, lời gọi API hệ điều hành, gửi dữ liệu đến máy chủ điều khiển (Command and Control - C2)... Dựa trên các hoạt động này, tập tin được phân loại là lành tính (benign) hoặc mã độc (malicious). Không giống phân tích tĩnh, phân tích động dựa trên sự tương tác thực tế giữa chương trình và hệ thống. Các tập tin PE chứa mã độc được thực thi trong môi trường ảo hóa được kiểm soát (sử dụng các công cụ như VirtualBox hoặc VMware) để tránh gây hại cho hệ thống thật.

**Phương pháp lai** (hybrid) kết hợp các yếu tố của cả phân tích tĩnh và phân tích động. Chẳng hạn, công cụ “HDM Analyser” [15, 16] sử dụng cả hai phương pháp trong giai đoạn huấn luyện (training phase), nhưng chỉ áp dụng phân tích tĩnh trong giai đoạn kiểm tra (testing phase) - nhằm tận dụng độ chính xác cao của phân tích động khi huấn luyện, đồng thời duy trì hiệu năng của phân tích tĩnh khi triển khai. So sánh cho thấy HDM Analyser đạt độ chính xác tổng thể và độ phức tạp thời gian tốt hơn so với các phương pháp chỉ dùng một trong hai loại phân tích.

Khó khăn cốt lõi của phương pháp lai là sự bất đối xứng về đặc trưng khả dụng

giữa hai giai đoạn: trong huấn luyện, cả đặc trưng tĩnh lẫn động đều có thể được thu thập đầy đủ trong môi trường kiểm soát, nhưng trong triển khai thực tế, đôi khi đặc trưng động không thể được thu thập đầy đủ vì không thể thực thi tùy tiện một file PE đáng ngờ trên hệ thống thật. Do đó, mô hình được huấn luyện với đặc trưng phong phú hơn so với những gì có thể cung cấp tại thời điểm triển khai (inference) - dẫn đến suy giảm hiệu năng trong vận hành thực tế. Để giải quyết vấn đề trên, có một số phương pháp như sau:

- **Feature distillation:** Huấn luyện mô hình nhận cả đặc trưng tĩnh và động, sau đó phân tích các đặc trưng tĩnh quan trọng nhất. Phương pháp này có thể gây mất thông tin nếu đặc trưng động mới là yếu tố quyết định (có feature importance lớn hơn).
- **Pseudo-dynamic features:** Tạo đặc trưng động giả bằng cách huấn luyện một mô hình phụ để dự đoán đặc trưng động từ đặc trưng tĩnh. Chất lượng dự đoán của kiến trúc tổng thể do đó phụ thuộc khá lớn vào độ chính xác của mô hình phụ.
- **Knowledge distillation:** Huấn luyện một mô hình giáo viên (teacher)  $T$  nhận đầu vào là các đặc trưng tĩnh và động, cho đầu ra là logits. Sử dụng  $T$  để tạo lại dữ liệu huấn luyện chỉ gồm đặc trưng tĩnh và các logits tương ứng, rồi dùng tập dữ liệu mới này để huấn luyện mô hình học sinh (student)  $S$  (nhận đầu vào là đặc trưng tĩnh, đầu ra là logits). Mục tiêu là  $S$  bắt chước đầu ra của  $T$  mà không cần đến đặc trưng động. Việc dùng logits thay vì nhãn cứng (hard labels) giúp  $S$  phân biệt được các ranh giới “mềm” giữa các lớp đầu ra, từ đó phân loại tốt hơn.

Ba phương pháp trên thể hiện các chiến lược khác nhau nhằm giải quyết bài toán mất cân bằng giữa khả năng phân biệt cao của đặc trưng động và yêu cầu hiệu năng, an toàn trong triển khai thực tế - vốn là ưu điểm của đặc trưng tĩnh. Feature distillation phù hợp với các hệ thống yêu cầu mô hình đơn giản, dễ diễn giải. Pseudo-dynamic features là lựa chọn tốt khi cần duy trì cấu trúc phân lớp ban đầu (chẳng hạn giữ nguyên chiều của vector đầu vào). Knowledge distillation là hướng tiếp cận tiên tiến hơn, giúp truyền tải hiệu quả phân biệt từ đặc trưng động sang mô hình gọn nhẹ, phù hợp với triển khai thực tế quy mô lớn.

Việc lựa chọn phương pháp không chỉ là bài toán kỹ thuật, mà còn phản ánh sự cân bằng giữa độ chính xác, hiệu năng và khả năng mở rộng - những yếu tố cốt lõi trong thiết kế hệ thống phát hiện mã độc hiện đại. Trong phạm vi khóa luận, để tập trung vào chất lượng mô hình và khả năng triển khai thực tế, *phương pháp phân tích tĩnh kết hợp với trích xuất năng lực (capability extraction)* được lựa chọn sử dụng. Trong đó, *capability extraction* là kỹ thuật xác định các hành vi và khả năng tiềm ẩn của file PE thông qua phân tích tĩnh mã nhị phân, mà không cần thực thi file. Tuy bản chất vẫn là phân tích tĩnh, *capability extraction* cho phép thu được thông tin mang tính ngữ nghĩa hành vi - vốn là ưu điểm đặc trưng của phân tích động - từ đó hình thành một hướng tiếp cận dung hòa giữa hai phương pháp: vừa duy trì được hiệu năng và độ an toàn của phân tích tĩnh, vừa bổ sung chiều sâu ngữ nghĩa mà các kỹ thuật phân tích tĩnh truyền thống thường không xem xét đầy đủ.

## 1.2 Tổng quan về ứng dụng học máy trong phát hiện mã độc

Phát hiện mã độc bằng học máy là quá trình sử dụng các thuật toán học máy để xây dựng mô hình phân loại, cho phép tự động phân biệt giữa phần mềm hợp lệ và mã độc dựa trên các đặc trưng được trích xuất từ dữ liệu. Khác với phương pháp dựa trên chữ ký, học máy không phụ thuộc hoàn toàn vào các mẫu mã độc đã biết, mà có khả năng khái quát hóa từ dữ liệu huấn luyện để phát hiện các biến thể mới hoặc các mẫu mã độc chưa từng xuất hiện trước đó.

Trong bối cảnh mã độc PE trên hệ điều hành Windows ngày càng được thiết kế tinh vi, thường xuyên sử dụng các kỹ thuật đóng gói, làm rối mã và né tránh phân tích, học máy cho phép khai thác mối quan hệ tiềm ẩn giữa các đặc trưng mà con người khó có thể xác định bằng các quy tắc thủ công. Các đặc trưng này có thể xuất phát từ cấu trúc file PE, hành vi gọi API, hoặc các biểu diễn trừu tượng hơn được học tự động bởi các mô hình học sâu.

Từ góc độ triển khai, một hệ thống phát hiện mã độc dựa trên học máy thường tuân theo một quy trình tổng quát, bao gồm: thu thập dữ liệu, phân tích và trích xuất đặc trưng, huấn luyện mô hình, và đánh giá hiệu quả. Việc áp dụng quy trình này giúp đảm bảo tính hệ thống, khả năng tái lập và tính khách quan trong đánh giá kết quả.

Liên quan đến dữ liệu đầu vào, học máy có thể được áp dụng cho nhiều hướng phân tích mã độc khác nhau, bao gồm phân tích tĩnh, phân tích động hoặc kết hợp cả hai. Bên cạnh đó, sự phát triển của học sâu đã mở ra khả năng tự động học biểu diễn đặc trưng từ dữ liệu thô, chẳng hạn như byte sequence hoặc opcode, thay vì phụ thuộc hoàn toàn vào đặc trưng được thiết kế thủ công. Các mô hình học sâu tận dụng ưu điểm này để nâng cao khả năng phát hiện các mẫu mã độc phức tạp, trong khi các mô hình học máy truyền thống như LightGBM lại thể hiện ưu thế về hiệu năng và tính ổn định khi triển khai.

Như vậy, phát hiện mã độc bằng học máy không chỉ là sự thay thế đơn thuần cho các phương pháp truyền thống, mà là một hướng tiếp cận mang tính tổng hợp, kết hợp kiến thức về hệ điều hành, định dạng PE, kỹ thuật phân tích mã độc và các thuật toán học máy hiện đại. Các phần tiếp theo sẽ lần lượt trình bày các hướng tiếp cận phổ biến, kỹ thuật trích xuất đặc trưng, và mô hình học máy được lựa chọn trong khuôn khổ khóa luận.

### **1.2.1 Các hướng tiếp cận phổ biến trong phát hiện mã độc bằng học máy**

Trong lĩnh vực phát hiện mã độc, đặc biệt là đối với mã độc định dạng PE trên hệ điều hành Windows, các phương pháp dựa trên học máy hiện nay có thể được phân loại theo nhiều hướng tiếp cận khác nhau, tùy thuộc vào loại dữ liệu đầu vào, cách trích xuất đặc trưng và mô hình học máy được sử dụng.

**Hướng tiếp cận dựa trên học máy truyền thống.** Học máy truyền thống là hướng tiếp cận sớm và vẫn được sử dụng rộng rãi trong phát hiện mã độc. Các mô hình tiêu biểu trong nhóm này bao gồm Logistic Regression, Support Vector Machine (SVM), Random Forest, XGBoost và LightGBM. Đặc điểm chung của các mô hình này là yêu cầu đặc trưng đầu vào được thiết kế thủ công, dựa trên kiến thức chuyên gia về cấu trúc file PE và hành vi của mã độc. Hiệu quả của mô hình do đó phụ thuộc lớn vào chất lượng và tính phân biệt của tập đặc trưng. Tuy nhiên, các mô hình học máy truyền thống thường có ưu điểm về thời gian huấn luyện và suy luận nhanh, khả năng diễn giải kết quả tương đối tốt, và dễ dàng triển khai trong các hệ thống thực tế.

**Hướng tiếp cận dựa trên học sâu.** Học sâu (Deep Learning) tận dụng các

mạng nơ-ron nhiều tầng để tự động học biểu diễn đặc trưng từ dữ liệu thô. Trong bài toán phát hiện mã độc PE, học sâu có thể được áp dụng trên nhiều dạng dữ liệu khác nhau như byte sequence, opcode, hoặc các biểu diễn trung gian của file PE. Ưu điểm chính là khả năng phát hiện các mẫu phức tạp và mối quan hệ phi tuyến trong dữ liệu, đặc biệt hiệu quả khi đối mặt với các kỹ thuật làm rối và biến thể mã độc. Tuy nhiên, học sâu thường đòi hỏi lượng dữ liệu huấn luyện lớn, tài nguyên tính toán cao và thời gian huấn luyện dài hơn so với học máy truyền thống.

**Nhận xét chung.** Mỗi hướng tiếp cận trong phát hiện mã độc bằng học máy đều có những ưu điểm và hạn chế riêng. Việc lựa chọn hướng tiếp cận phù hợp cần cân nhắc giữa độ chính xác, hiệu năng và khả năng triển khai trong thực tế. Trong đề tài này, hướng tiếp cận chính là học máy truyền thống kết hợp phân tích tĩnh, sử dụng mô hình LightGBM làm nền tảng. Trọng tâm thực nghiệm không nằm ở việc so sánh các loại mô hình, mà ở việc đánh giá hiệu quả của các tổ hợp đặc trưng khác nhau - cụ thể là đặc trưng EMBER2024 thuần túy, kết hợp với đặc trưng năng lực (capability) trích xuất bằng công cụ Capa ở dạng one-hot và dạng đã giảm chiều bằng TruncatedSVD - nhằm xác định tổ hợp đặc trưng tối ưu cho bài toán phát hiện mã độc PE.

### 1.2.2 Trích xuất đặc trưng từ file PE

Trong bài toán phát hiện mã độc dựa trên học máy, quá trình trích xuất đặc trưng đóng vai trò then chốt, quyết định trực tiếp đến hiệu quả của mô hình. Đối với các file thực thi định dạng PE, các đặc trưng có thể được trích xuất từ nhiều khía cạnh khác nhau, phản ánh cấu trúc, nội dung và một phần hành vi tiềm ẩn của chương trình. Phần này trình bày các nhóm đặc trưng chính được sử dụng trong đề tài, đặc biệt là trong các feature extractor (bộ trích xuất đặc trưng) nổi tiếng như EMBERv2 và EMBER2024 [17].

#### Đặc trưng tổng quát của file PE

Nhóm đặc trưng này phản ánh các thông tin tổng quát của file PE, bao gồm kích thước file, số lượng section, thời gian biên dịch (timestamp) và các cờ (flag) cấu hình trong header. Các đặc trưng này giúp mô hình học máy nắm bắt được những khác biệt cơ bản giữa file thực thi hợp lệ và mã độc, bởi mã độc thường có cấu trúc

bất thường hoặc được tạo ra thông qua các công cụ tự động như packer, obfuscator, cryptor...

## **Đặc trưng cấu trúc PE và header**

Các đặc trưng cấu trúc được trích xuất từ các thành phần quan trọng trong file PE như DOS header, PE header và optional header. Nhóm đặc trưng này bao gồm các trường thông tin phản ánh cách thức file được nạp và thực thi trên hệ điều hành Windows. Những sai lệch hoặc giá trị bất thường trong các trường header thường là dấu hiệu của mã độc hoặc các file đã bị chỉnh sửa.

## **Đặc trưng liên quan đến section**

Mỗi file PE được chia thành nhiều section với các mục đích khác nhau. Các đặc trưng liên quan đến section bao gồm số lượng section, kích thước của từng section, quyền truy cập (read, write, execute) và phân bố entropy giữa các section. Đặc biệt, entropy cao ở một số section có thể là dấu hiệu của kỹ thuật đóng gói hoặc mã hóa, vốn thường được sử dụng trong mã độc để né tránh phân tích.

Trong lý thuyết thông tin, entropy được sử dụng như một thước đo mức độ hỗn loạn và tính ngẫu nhiên của dữ liệu. Đối với các chương trình lành tính thông thường, dữ liệu trong các section của file thực thi thường có cấu trúc tương đối rõ ràng; khi tiến hành dịch ngược, có thể quan sát được luồng điều khiển và logic chương trình ở mức cơ bản. Ngược lại, nhiều loại mã độc sử dụng các kỹ thuật đóng gói hoặc mã hóa, làm cho nội dung các section trở nên khó phân tích và có mức entropy cao hơn so với các file thực thi thông thường.

Tuy nhiên, cần lưu ý rằng entropy cao không phải là dấu hiệu đặc trưng tuyệt đối của mã độc. Một số phần mềm thương mại cũng áp dụng các kỹ thuật bảo vệ mã nguồn tương tự, dẫn đến mức entropy cao ở một số section. Do đó, việc đánh giá mã độc chỉ dựa trên entropy là không đủ và cần kết hợp với các nhóm đặc trưng khác để đưa ra quyết định phân loại chính xác hơn.

## **Đặc trưng về bảng import và export**

Nhóm đặc trưng này tập trung vào các hàm và thư viện được file PE sử dụng. Việc phân tích các thư viện và hàm import cho phép suy đoán hành vi tiềm ẩn của chương trình, chẳng hạn như thao tác với hệ thống file, registry, mạng hoặc tiến trình. Các file mã độc thường sử dụng các hàm API đặc trưng phục vụ cho việc ẩn mình, duy trì tồn tại hoặc thu thập thông tin.

## **Đặc trưng thống kê từ nội dung nhị phân**

Nhóm đặc trưng thống kê được trích xuất trực tiếp từ nội dung nhị phân của file, bao gồm phân bố byte, tần suất xuất hiện của các giá trị byte và các thống kê liên quan. Những đặc trưng này phản ánh cấu trúc tổng thể của file ở mức thấp và có khả năng phân biệt giữa file hợp lệ và mã độc ngay cả khi mã đã bị làm rối.

## **Đặc trưng trích xuất từ chuỗi văn bản**

Trong file PE, nhiều chuỗi văn bản có thể được trích xuất, chẳng hạn như tên file, đường dẫn, URL, tên hàm hoặc thông báo lỗi. Việc phân tích các chuỗi này giúp phát hiện các dấu hiệu liên quan đến hoạt động độc hại, như kết nối mạng, tải thêm mã hoặc tương tác với hệ thống. Các chuỗi văn bản thường được mã độc sử dụng để phục vụ cho quá trình thực thi hoặc giao tiếp với máy chủ điều khiển.

## **Đặc trưng dạng biểu diễn rút gọn**

Để phù hợp với các mô hình học máy truyền thống, nhiều đặc trưng có tính chất danh mục hoặc chuỗi được chuyển đổi sang dạng biểu diễn số thông qua các kỹ thuật rút gọn và mã hóa. Các kỹ thuật này giúp giảm chiều dữ liệu, đồng thời vẫn giữ lại các thông tin quan trọng phục vụ cho quá trình học của mô hình.

## **Đặc trưng dựa trên chữ ký số**

Trong hệ điều hành Windows, các file thực thi PE có thể được ký số nhằm xác thực nguồn gốc và đảm bảo tính toàn vẹn của file. Các đặc trưng liên quan đến chữ ký số phản ánh trạng thái xác thực của file, chẳng hạn như việc file có được



ký số hay không, chữ ký có hợp lệ hay không, cũng như một số thuộc tính cơ bản của chứng thư số đi kèm.

Trong thực tế, phần lớn các phần mềm hợp lệ thường được ký số để tăng độ tin cậy, trong khi nhiều mẫu mã độc không được ký số hoặc sử dụng chữ ký không hợp lệ, tự ký hoặc đã bị chỉnh sửa. Tuy nhiên, sự tồn tại của chữ ký số hợp lệ không đảm bảo tuyệt đối file là an toàn, bởi mã độc vẫn có thể được phát tán thông qua các chứng thư bị lạm dụng hoặc bị đánh cắp. Vì vậy, nhóm đặc trưng này được sử dụng như một yếu tố bổ trợ, kết hợp với các nhóm đặc trưng khác trong quá trình phân loại.

### **Đặc trưng tổng hợp các lỗi đọc file PE**

Quá trình phân tích tĩnh file PE đòi hỏi việc đọc và diễn giải chính xác cấu trúc nội bộ của file theo định dạng chuẩn. Trong thực tế, nhiều file mã độc được tạo ra với cấu trúc bất thường, bị làm hỏng có chủ đích hoặc cố tình vi phạm chuẩn định dạng nhằm gây khó khăn cho các công cụ phân tích. Do đó, các lỗi phát sinh khi đọc hoặc phân tích cấu trúc file PE - như header không hợp lệ, bảng section bị thiếu hoặc trùng lặp, offset không nhất quán, hoặc các trường dữ liệu chứa giá trị bất thường - có thể được ghi nhận và tổng hợp thành một nhóm đặc trưng riêng.

Về mặt thống kê, một file thực thi phát sinh càng nhiều lỗi trong quá trình phân tích thì mức độ bất thường của file càng cao, thường phản ánh việc file đã bị chỉnh sửa, làm rối hoặc được tạo ra bởi các công cụ tự động - những đặc điểm phổ biến của mã độc. Tuy nhiên, tương tự như các nhóm đặc trưng khác, lỗi phân tích không thể được sử dụng độc lập để kết luận một file là mã độc, bởi một số file hợp lệ cũng có thể bị hỏng hoặc không tuân thủ hoàn toàn chuẩn định dạng. Vì vậy, nhóm đặc trưng này được kết hợp với các đặc trưng cấu trúc, thống kê và nội dung khác nhằm nâng cao độ chính xác của mô hình.

Ngoài các nhóm đặc trưng nêu trên, một số hướng mở rộng có thể được xem xét trong các nghiên cứu tiếp theo, bao gồm đặc trưng dựa trên opcode hoặc lệnh máy (assembly), đặc trưng dựa trên đồ thị luồng điều khiển, và đặc trưng kết hợp giữa phân tích tĩnh và động. Tuy nhiên, trong phạm vi đề tài, các nhóm đặc trưng tĩnh từ file PE được lựa chọn nhằm đảm bảo tính an toàn, khả năng mở rộng và phù hợp với yêu cầu thực nghiệm.

### 1.2.3 Một số mô hình học máy tiêu biểu trong phát hiện mã độc PE

Trong những năm gần đây, nhiều mô hình học máy đã được nghiên cứu và áp dụng vào bài toán phát hiện mã độc, đặc biệt là đối với các file thực thi định dạng PE trên hệ điều hành Windows. Tùy thuộc vào đặc điểm dữ liệu, phương pháp trích xuất đặc trưng và yêu cầu triển khai, các mô hình học máy khác nhau thể hiện những ưu điểm và hạn chế riêng.

**Hồi quy Logistic** là một trong những mô hình học máy đơn giản và phổ biến nhất cho bài toán phân loại nhị phân. Trong phát hiện mã độc, mô hình này được sử dụng để ước lượng xác suất một file thực thi là mã độc dựa trên các đặc trưng đầu vào. Ưu điểm của hồi quy Logistic là dễ cài đặt, tốc độ huấn luyện nhanh và khả năng diễn giải cao. Tuy nhiên, do giả định mối quan hệ tuyến tính giữa đặc trưng và nhãn, mô hình này thường gặp hạn chế khi xử lý dữ liệu phức tạp và có tính phi tuyến cao như trong các tập dữ liệu mã độc thực tế.

**Support Vector Machine (SVM)** là mô hình học máy mạnh mẽ trong việc phân loại dữ liệu có số chiều lớn. SVM hoạt động bằng cách tìm siêu phẳng tối ưu nhằm phân tách các lớp dữ liệu với khoảng cách lớn nhất. Trong bài toán phát hiện mã độc, SVM cho kết quả tương đối tốt và tỏ ra vượt trội hơn hồi quy logistic khi sử dụng các kernel phi tuyến. Tuy nhiên, chi phí tính toán của SVM có thể tăng cao khi kích thước tập dữ liệu lớn, khiến việc triển khai trong môi trường thực tế gặp nhiều hạn chế.

**Decision Tree và Random Forest.** Cây quyết định (Decision Tree) là mô hình học máy dựa trên cấu trúc cây, trong đó mỗi nút đại diện cho một điều kiện phân tách dữ liệu. Mô hình này có ưu điểm là trực quan, dễ hiểu và có khả năng xử lý dữ liệu phi tuyến. Rừng ngẫu nhiên (Random Forest) là mô hình mở rộng của cây quyết định, kết hợp nhiều cây độc lập để nâng cao độ chính xác và giảm hiện tượng quá khớp. Trong phát hiện mã độc, rừng ngẫu nhiên thường được sử dụng nhờ khả năng xử lý tốt dữ liệu nhiều chiều và tính ổn định cao.

**Các mô hình tăng cường dựa trên cây** như Gradient Boosting, XGBoost và LightGBM đã cho thấy hiệu quả vượt trội trong nhiều bài toán phân loại, bao gồm phát hiện mã độc. Các mô hình này xây dựng tập hợp các cây yếu theo cách tuần tự, trong đó mỗi cây mới tập trung khắc phục các sai sót của các cây trước đó. Ưu điểm của nhóm mô hình này là khả năng học các mối quan hệ phi tuyến phức tạp,

xử lý hiệu quả dữ liệu có số chiều lớn và khả năng kiểm soát hiện tượng quá khớp thông qua các tham số điều chỉnh. Trong đề tài này, LightGBM được lựa chọn để triển khai và đánh giá chi tiết hơn.

Các mô hình học máy truyền thống, đặc biệt là các mô hình dựa trên cây và tăng cường, vẫn đóng vai trò quan trọng trong phát hiện mã độc nhờ sự cân bằng giữa độ chính xác, hiệu năng và khả năng triển khai. Việc lựa chọn mô hình phù hợp phụ thuộc vào đặc điểm tập dữ liệu, nhóm đặc trưng được sử dụng và yêu cầu cụ thể của bài toán.

#### 1.2.4 Mô hình LightGBM

Light Gradient Boosting Machine (LightGBM) là một mô hình học máy thuộc nhóm các thuật toán tăng cường dựa trên cây quyết định, được thiết kế nhằm nâng cao hiệu quả huấn luyện và khả năng mở rộng đối với các tập dữ liệu có kích thước lớn và số lượng đặc trưng cao. Trong bối cảnh bài toán phát hiện mã độc PE, LightGBM cho thấy nhiều ưu điểm phù hợp với yêu cầu thực nghiệm và triển khai.

LightGBM xây dựng mô hình bằng cách kết hợp nhiều cây quyết định yếu theo cơ chế học tăng cường, trong đó mỗi cây mới được huấn luyện nhằm giảm sai số còn tồn tại của các cây trước đó. Khác với các phương pháp tăng cường truyền thống, LightGBM áp dụng chiến lược tăng trưởng cây theo chiều sâu (leaf-wise), giúp mô hình hội tụ nhanh hơn và khai thác hiệu quả các mối quan hệ phi tuyến trong dữ liệu.

Một ưu điểm quan trọng của LightGBM là khả năng xử lý hiệu quả dữ liệu có số chiều lớn, vốn là đặc trưng phổ biến trong bài toán phát hiện mã độc khi sử dụng nhiều nhóm đặc trưng trích xuất từ file PE. Bên cạnh đó, LightGBM hỗ trợ cơ chế rời rạc hóa đặc trưng và tối ưu bộ nhớ, giúp giảm chi phí tính toán trong quá trình huấn luyện và suy luận.

Trong phạm vi đề tài, LightGBM được lựa chọn làm mô hình đại diện cho hướng tiếp cận học máy truyền thống nhờ các đặc điểm sau:

- Khả năng đạt độ chính xác cao với dữ liệu đặc trưng tĩnh từ file PE.
- Thời gian huấn luyện và suy luận nhanh, phù hợp với kịch bản xử lý số lượng

lớn file.

- Tính ổn định và khả năng kiểm soát hiện tượng quá khớp thông qua các tham số điều chỉnh.

Ngoài ra, LightGBM cho phép đánh giá mức độ đóng góp của từng đặc trưng thông qua các chỉ số feature importance hoặc kỹ thuật xAI như SHAP, giúp hỗ trợ phân tích và giải thích kết quả thực nghiệm. Khả năng này đặc biệt hữu ích trong việc đánh giá vai trò của từng nhóm đặc trưng, cung cấp cơ sở lý giải cho kết quả thực nghiệm thu được.

## 1.3 Các công trình nghiên cứu liên quan

Phần này điểm qua các công trình nghiên cứu và tập dữ liệu tiêu biểu liên quan đến bốn khía cạnh chính của khóa luận: tập dữ liệu phục vụ huấn luyện mô hình, ứng dụng học máy trong phân tích tĩnh file PE, kỹ thuật trích xuất năng lực mã độc, và phương pháp lý giải quyết định của mô hình. Trên cơ sở đó, khóa luận xác định điểm kế thừa và đóng góp riêng so với các nghiên cứu hiện có.

### 1.3.1 Các tập dữ liệu file PE phục vụ huấn luyện mô hình học máy phát hiện mã độc

Chất lượng và quy mô tập dữ liệu có ảnh hưởng trực tiếp đến hiệu quả huấn luyện và khả năng tổng quát hóa của mô hình học máy trong bài toán phát hiện mã độc. Một số tập dữ liệu tiêu biểu đã được công bố và sử dụng rộng rãi trong cộng đồng nghiên cứu.

**EMBER (Endgame Malware BENCHMARK for Research).** EMBER [5] là một trong những tập dữ liệu benchmark được sử dụng phổ biến nhất cho bài toán phát hiện mã độc PE. Phiên bản đầu tiên được Anderson và Roth công bố năm 2018, bao gồm khoảng 1,1 triệu mẫu file PE (lành tính, mã độc và chưa gán nhãn), với các đặc trưng tĩnh được trích xuất sẵn theo định dạng vector có cấu trúc. Tập dữ liệu này cung cấp một pipeline trích xuất đặc trưng chuẩn hóa, bao gồm các nhóm đặc trưng từ PE header, section, import table, chuỗi văn bản và nội dung nhị phân, giúp tạo điều kiện thuận lợi cho việc so sánh các mô hình học máy trên cùng một nền tảng dữ liệu.

Phiên bản EMBER2024 [17] là bản cập nhật mới nhất, được xây dựng nhằm phản ánh bối cảnh mã độc hiện đại hơn so với phiên bản 2018. EMBER2024 bổ sung số lượng mẫu lớn hơn, cập nhật phân bố thời gian thu thập, và mở rộng pipeline trích xuất đặc trưng. Trong đề tài này, EMBER2024 được lựa chọn làm tập dữ liệu chính cho tất cả các thực nghiệm.

**VirusShare và VirusTotal.** VirusShare [18] là một kho lưu trữ mẫu mã độc lớn được duy trì bởi cộng đồng nghiên cứu bảo mật, cung cấp hàng triệu mẫu file thực thi đã được xác nhận là mã độc. VirusTotal [19] là dịch vụ phân tích file trực tuyến tổng hợp kết quả từ hàng chục engine antivirus, thường được dùng để gán nhãn hoặc xác minh nhãn cho các mẫu trong quá trình xây dựng tập dữ liệu. Nhiều nghiên cứu sử dụng kết hợp hai nguồn này để thu thập mẫu mã độc đa dạng và đảm bảo độ tin cậy của nhãn phân loại.

**Các tập dữ liệu khác.** Ngoài EMBER, một số tập dữ liệu khác cũng được sử dụng trong các nghiên cứu liên quan, chẳng hạn như SOREL-20M [20] - tập dữ liệu quy mô lớn với 20 triệu mẫu PE do Sophos và ReversingLabs công bố - cũng như các tập dữ liệu tự thu thập từ honeypot hoặc các nguồn chia sẻ trong cộng đồng bảo mật. Tuy nhiên, EMBER vẫn là lựa chọn phổ biến nhất nhờ tính chuẩn hóa, khả năng tái lập và sự hỗ trợ, công nhận rộng rãi của cộng đồng nghiên cứu.

### 1.3.2 Các nghiên cứu về ứng dụng học máy trong phân tích tĩnh file thực thi nhị phân

Ứng dụng học máy trong phân tích tĩnh file PE là một hướng nghiên cứu được quan tâm rộng rãi, với nhiều công trình tiêu biểu đã đạt kết quả đáng chú ý.

**Các mô hình học máy truyền thống trên đặc trưng tĩnh.** Anderson và Roth [5] đã đề xuất mô hình LightGBM huấn luyện trên tập đặc trưng EMBER, đạt tỷ lệ phát hiện mã độc cao với tốc độ suy luận phù hợp cho triển khai thực tế. Đây là một trong những baseline quan trọng nhất trong lĩnh vực, cho thấy các mô hình tăng cường dựa trên cây quyết định có thể đạt hiệu quả cạnh tranh với các phương pháp phức tạp hơn khi được kết hợp với bộ đặc trưng tĩnh chất lượng cao.

**Kỹ thuật giảm chiều trong xử lý đặc trưng.** Khi tập đặc trưng có số chiều lớn - đặc biệt khi kết hợp nhiều nhóm đặc trưng khác nhau - các kỹ thuật giảm chiều như PCA (Principal Component Analysis) [21] và TruncatedSVD [22] thường được

áp dụng để giảm chi phí tính toán và hạn chế hiện tượng quá khớp. TruncatedSVD đặc biệt phù hợp với các ma trận đặc trưng thưa (sparse), vốn xuất hiện phổ biến khi mã hóa đặc trưng dạng one-hot hoặc bag-of-words từ dữ liệu mã độc.

**Phương pháp Ensemble.** Các phương pháp kết hợp mô hình (ensemble) như voting, stacking và blending [23] đã được áp dụng trong phát hiện mã độc nhằm kết hợp điểm mạnh của nhiều mô hình thành phần khác loại (heterogeneous ensemble), từ đó nâng cao độ chính xác và tính ổn định tổng thể. Tuy nhiên, cần phân biệt rõ hai cấp độ ensemble: ensemble giữa các mô hình khác loại (ví dụ kết hợp LightGBM với SVM hay mạng neuron) và ensemble nội tại bên trong một mô hình. LightGBM thuộc nhóm thứ hai - bản thân mô hình đã là một tổ hợp của hàng trăm cây quyết định được xây dựng tuần tự theo cơ chế gradient boosting, trong đó mỗi cây mới tập trung khắc phục sai số của các cây trước. Cơ chế này mang lại khả năng học các mối quan hệ phi tuyến phức tạp và tính ổn định cao, mà không cần thêm một lớp ensemble bên ngoài. Các nghiên cứu thực nghiệm [5, 20] cho thấy LightGBM đơn lẻ đã đạt hiệu năng cạnh tranh với các heterogeneous ensemble phức tạp hơn trong bài toán phát hiện mã độc PE, trong khi vẫn duy trì ưu thế về tốc độ suy luận và khả năng triển khai thực tế - những yếu tố đặc biệt quan trọng khi triển khai mô hình ngay trong dịch vụ chạy trên máy endpoint trong khuôn khổ khóa luận.

### 1.3.3 Các nghiên cứu về trích xuất năng lực (capability extraction) của file thực thi nhị phân

Trích xuất năng lực (capability extraction) là kỹ thuật xác định các hành vi và khả năng tiềm ẩn của một file thực thi thông qua phân tích tĩnh, mà không cần thực thi file trong môi trường thật. Đây là hướng tiếp cận có tiềm năng cao trong việc bổ sung thông tin ngữ nghĩa hành vi vào quá trình phát hiện mã độc.

**Công cụ Capa.** Capa [24] là công cụ mã nguồn mở do MANDIANT (trước đây là FireEye) phát triển, cho phép tự động nhận diện các năng lực của file thực thi nhị phân thông qua một tập quy tắc (rule) được xây dựng thủ công bởi các chuyên gia phân tích mã độc. Mỗi quy tắc trong Capa mô tả một năng lực cụ thể - chẳng hạn “mã hóa dữ liệu bằng XOR”, “tạo tiến trình mới”, hay “kết nối đến địa chỉ IP từ xa” - và được ánh xạ sang các framework phân loại hành vi chuẩn như MITRE ATT&CK [25] và MBC (Malware Behavior Catalog) [26]. Capa hỗ trợ phân tích

file PE thông qua kỹ thuật phân tích tĩnh disassembly (dịch ngược hợp ngữ), sử dụng các backend như SMDA hoặc IDA Pro để dịch ngược và đối chiếu với tập quy tắc.

Trong đề tài này, Capa được sử dụng để trích xuất vector đặc trưng năng lực từ các file PE trong tập EMBER2024, sau đó kết hợp với đặc trưng EMBER2024 gốc để huấn luyện mô hình học máy. Điểm khác biệt so với cách dùng Capa truyền thống (chủ yếu phục vụ phân tích thủ công) là đề tài tự động hóa quá trình này ở quy mô lớn và tích hợp kết quả vào pipeline học máy.

Một số nghiên cứu đã khảo sát việc sử dụng thông tin về năng lực và hành vi của mã độc như một nguồn đặc trưng bổ sung cho mô hình học máy [27, 28]. Các kết quả ban đầu cho thấy đặc trưng năng lực có tính phân biệt cao, đặc biệt trong việc phân loại họ mã độc và phát hiện các biến thể mới sử dụng kỹ thuật obfuscation. Tuy nhiên, việc tích hợp đặc trưng Capa vào pipeline học máy ở quy mô lớn vẫn còn ít được nghiên cứu một cách có hệ thống, đặc biệt khi kết hợp với các kỹ thuật giảm chiều như TruncatedSVD - đây là một trong những đóng góp chính của khóa luận.

#### 1.3.4 Các nghiên cứu về lý giải quyết định của mô hình (Explainable AI - xAI)

Khi các mô hình học máy ngày càng được triển khai trong các hệ thống bảo mật thực tế, khả năng lý giải quyết định của mô hình (Explainable AI - xAI) trở thành yêu cầu quan trọng, giúp chuyên gia bảo mật hiểu được lý do một file bị phân loại là mã độc và đưa ra quyết định phản ứng phù hợp.

**SHAP (SHapley Additive exPlanations).** SHAP [29] là một trong những phương pháp xAI phổ biến và có nền tảng lý thuyết vững chắc nhất hiện nay, dựa trên khái niệm Shapley value từ lý thuyết trò chơi hợp tác. SHAP tính toán mức độ đóng góp của từng đặc trưng vào kết quả dự đoán của mô hình cho từng mẫu cụ thể, cho phép giải thích quyết định ở cấp độ cục bộ (per-instance) lẫn toàn cục (global feature importance). Trong bài toán phát hiện mã độc, SHAP được sử dụng để xác định những đặc trưng nào - chẳng hạn một năng lực Capa cụ thể hay một trường PE header - đóng vai trò quyết định trong việc phân loại một file là mã độc.

**LIME (Local Interpretable Model-agnostic Explanations).** LIME [30] là phương pháp xAI không phụ thuộc vào loại mô hình (model-agnostic), hoạt động bằng cách xây dựng một mô hình tuyến tính đơn giản xấp xỉ hành vi của mô hình phức tạp trong vùng lân cận của một điểm dữ liệu cụ thể. LIME phù hợp với các trường hợp cần giải thích nhanh và trực quan cho từng mẫu riêng lẻ, mặc dù độ ổn định của giải thích có thể thấp hơn so với SHAP trong một số trường hợp.

**Ứng dụng xAI trong phát hiện mã độc.** Một số nghiên cứu đã kết hợp SHAP với các mô hình phát hiện mã độc dựa trên học máy [31], cho thấy xAI không chỉ hỗ trợ kiểm tra và gỡ lỗi mô hình mà còn cung cấp thông tin hữu ích cho quá trình điều tra và ứng phó sự cố. Trong đề tài này, SHAP được tích hợp vào server phân tích sâu (deep-analysis-server) nhằm cung cấp giải thích trực quan cho kết quả phân loại, giúp người dùng hiểu được những đặc trưng nào - bao gồm cả đặc trưng Capa - đã dẫn đến quyết định phân loại của mô hình.

**Nhận xét chung.** So với các công trình liên quan, đề tài này có những điểm kế thừa và khác biệt như sau. Về kế thừa, đề tài sử dụng tập dữ liệu EMBER2024 và mô hình LightGBM theo hướng đã được thiết lập tốt trong cộng đồng nghiên cứu, đồng thời áp dụng SHAP - một kỹ thuật xAI đã được kiểm chứng - để giải thích kết quả. Về đóng góp riêng, đề tài tập trung vào việc tích hợp có hệ thống đặc trưng năng lực từ Capa vào pipeline học máy ở quy mô lớn, đánh giá định lượng mức độ đóng góp của đặc trưng này so với baseline EMBER2024 thuần túy, và triển khai pipeline vào một dịch vụ bảo vệ endpoint thực tế trên Windows.

## 1.4 Công nghệ sử dụng

Đề tài kết hợp nhiều công nghệ và công cụ thuộc các lĩnh vực khác nhau, từ học máy và phân tích mã độc đến lập trình hệ thống Windows. Phần này trình bày tổng quan các công nghệ chính được sử dụng trong quá trình thực hiện.

### 1.4.1 Ngôn ngữ lập trình và công cụ xây dựng

**Python** là ngôn ngữ lập trình chính cho các thành phần liên quan đến học máy và phân tích dữ liệu. Python được lựa chọn nhờ hệ sinh thái thư viện phong phú, đặc biệt trong lĩnh vực khoa học dữ liệu và học máy, cùng với khả năng tích hợp nhanh chóng với các công cụ phân tích mã độc hiện có.



C++ được sử dụng cho module trích xuất đặc trưng EMBER2024 chạy trực tiếp trên máy endpoint. Lựa chọn này xuất phát từ yêu cầu về hiệu năng - việc trích xuất đặc trưng cần được thực hiện nhanh chóng trên từng file PE trong thời gian thực - cũng như khả năng tích hợp tự nhiên với các API hệ thống Windows. Dự án C++ được quản lý bằng **CMake**, công cụ build đa nền tảng phổ biến, giúp đảm bảo tính nhất quán trong quá trình biên dịch và triển khai.

#### 1.4.2 Lập trình hệ thống Windows

Các thành phần chạy trực tiếp trên máy endpoint được xây dựng dựa trên các công nghệ lập trình hệ thống của Windows.

**Windows API** [32] là tập hợp các hàm và giao diện lập trình do Microsoft cung cấp, cho phép ứng dụng tương tác trực tiếp với hệ điều hành Windows ở mức người dùng (user mode). Trong đề tài, Windows API được sử dụng để xây dựng dịch vụ nền (background service) và giao diện người dùng (UI) chạy trên endpoint, bao gồm các chức năng quản lý tiến trình, giao tiếp liên tiến trình (IPC), tương tác với hệ thống file, cùng một số chức năng khác.

**Kernel-Mode Driver Framework (KMDF)** [33] là framework do Microsoft cung cấp để phát triển driver chạy ở chế độ nhân (kernel mode) trên Windows. KMDF cung cấp một lớp trừu tượng trên WDM (Windows Driver Model), giúp đơn giản hóa việc lập trình driver và giảm thiểu rủi ro gây mất ổn định hệ thống. Trong đề tài, KMDF được sử dụng để xây dựng thành phần driver có khả năng theo dõi sự kiện thực thi file ở tầng nhân, phục vụ cho cơ chế phát hiện mã độc theo thời gian thực.

#### 1.4.3 Phát triển dịch vụ phân tích

**FastAPI** [34] là framework Python hiện đại để xây dựng REST API, nổi bật với hiệu năng cao nhờ hỗ trợ bất đồng bộ (async) và khả năng tự động sinh tài liệu API. FastAPI được sử dụng làm nền tảng cho server phân tích sâu, tiếp nhận yêu cầu phân tích file từ các thành phần khác trong hệ thống và trả về kết quả phân loại cùng giải thích từ mô hình.

Giao diện web của server phân tích được xây dựng bằng **HTML, CSS và JavaScript** thuần, không phụ thuộc vào framework frontend phức tạp, nhằm đảm bảo tính đơn

giản và dễ triển khai.

**Docker** [35] được sử dụng để đóng gói và triển khai server phân tích sâu dưới dạng container, đảm bảo tính nhất quán về môi trường thực thi và đơn giản hóa quá trình cài đặt trên các máy khác nhau.

#### 1.4.4 Thư viện học máy và phân tích mã độc

**LightGBM** [36] là thư viện gradient boosting hiệu năng cao do Microsoft phát triển, được sử dụng làm mô hình phân loại chính trong đề tài. LightGBM hỗ trợ huấn luyện song song, tối ưu bộ nhớ và cung cấp API tích hợp tốt với hệ sinh thái Python. Ngoài ra, thư viện còn có phiên bản (binding) ngôn ngữ C++, hỗ trợ việc triển khai mô hình trực tiếp trên máy endpoint, đảm bảo hiệu năng mà không cần Python runtime.

**scikit-learn** [37] là thư viện học máy nền tảng trong Python, được sử dụng cho các tác vụ tiền xử lý dữ liệu, giảm chiều đặc trưng (TruncatedSVD) và đánh giá mô hình.

**SHAP** [29] là thư viện Python cung cấp các phương pháp giải thích mô hình học máy dựa trên Shapley value. SHAP được tích hợp vào server phân tích sâu để tạo ra các giải thích trực quan về lý do mô hình phân loại một file là mã độc.

**Capa** [24] là công cụ mã nguồn mở do Mandiant phát triển, cho phép tự động nhận diện năng lực (capability) của file thực thi nhị phân thông qua phân tích tĩnh dựa trên tập quy tắc. Trong đề tài, Capa được tích hợp vào pipeline trích xuất đặc trưng để bổ sung thông tin ngữ nghĩa hành vi vào quá trình huấn luyện mô hình, cũng như được sử dụng trong dịch vụ phân tích chuyên sâu online (deep-analysis-server).

| Tên           | Kiến trúc | Truyền tham số<br>( <i>kiểu số nguyên hoặc pointer</i> )         | Trách nhiệm clean up stack | Ghi chú                                                                          |
|---------------|-----------|------------------------------------------------------------------|----------------------------|----------------------------------------------------------------------------------|
| cdecl         | x86       | Đẩy tham số lên stack (tham số cuối được đẩy trước)              | Caller                     | Dùng phổ biến trong C                                                            |
| stdcall       | x86       | Như cdecl                                                        | <i>Callee</i>              | Dùng trong WINAPI                                                                |
| fastcall      | x86       | ECX, EDX, và đẩy các tham số còn lại lên stack như cdecl         | <i>Callee</i>              | Nhanh hơn, thường thấy trong tối ưu (compiler optimization), song không phổ biến |
| Microsoft x64 | amd64     | RCX, RDX, R8, R9, và đẩy các tham số còn lại lên stack như cdecl | Caller                     | Mặc định trên Windows 64-bit                                                     |

Bảng 1.1: Các calling convention phổ biến trên Windows

# Chương 2

## Thiết kế và đánh giá các pipeline học máy trong phát hiện mã độc

### 2.1 Tổng quan kiến trúc giải pháp

Ở Chương 1, đề tài đã trình bày các nhóm đặc trưng tĩnh thường được trích xuất từ file PE - bao gồm thông tin header, section, import table, chuỗi văn bản và nội dung nhị phân - và cho thấy đây là nền tảng hiệu quả để xây dựng mô hình phát hiện mã độc. Tuy nhiên, các đặc trưng này chủ yếu phản ánh *cấu trúc* của file PE, trong khi thông tin về *hành vi* và *năng lực* tiềm ẩn của file - chẳng hạn khả năng mã hóa dữ liệu, leo thang đặc quyền hay kết nối đến máy chủ điều khiển - vẫn chưa được khai thác. Chương này đề xuất bổ sung loại thông tin trên thông qua kỹ thuật trích xuất năng lực (capability extraction) bằng công cụ Capa, kết hợp với bộ đặc trưng EMBER2024, đồng thời đánh giá liệu sự bổ sung này có giúp cải thiện hiệu quả phân loại mã độc PE hay không.

Cụ thể, chương này xây dựng và so sánh **03 pipeline** học máy, có sự khác biệt về cách xây dựng vector đặc trưng đầu vào, trong khi giữ cố định mô hình (LightGBM), bộ siêu tham số (hyperparameters), tập dữ liệu và chiến lược chia tập train/test (Hình 2.1):

- **Pipeline A:** chỉ sử dụng đặc trưng EMBER2024 (baseline).
- **Pipeline B:** EMBER2024 kết hợp đặc trưng Capa dạng multi-hot encoding.
- **Pipeline C:** EMBER2024 kết hợp đặc trưng Capa sau khi giảm chiều bằng TruncatedSVD, thử nghiệm với các số chiều 64, 128 và 256.

Với mỗi file PE đầu vào, pipeline luôn thực hiện bước trích xuất đặc trưng EMBER2024 tạo ra vector 2.568 chiều. Tùy từng pipeline, có thể triển khai bước trích xuất năng lực (capabilities) của các file PE bằng Capa, chuyển đổi chúng thành vector bằng một phương pháp nhất định, rồi ghép nối với vector EMBER2024

trước khi đưa vào LightGBM để phân loại. Sự khác biệt giữa các pipeline nằm hoàn toàn ở bước xây dựng vector đặc trưng, giúp cô lập và đo lường chính xác đóng góp của từng nhóm đặc trưng.

Lựa chọn **LightGBM** làm mô hình phân loại xuất phát từ các lý do đã trình bày ở Chương 1: hiệu năng cao trên dữ liệu đặc trưng tĩnh nhiều chiều, tốc độ suy luận nhanh phù hợp với triển khai thực tế, và khả năng diễn giải thông qua feature importance. Bản thân LightGBM đã là một phương pháp ensemble nội tại (gradient boosting của nhiều cây quyết định), do đó không cần thêm lớp ensemble bên ngoài. Đáng chú ý, mô hình LightGBM pretrained do nhóm tác giả EMBER2024 cung cấp - được huấn luyện trên toàn bộ tập dữ liệu gốc - đạt ROC-AUC tổng thể là 0,9949, cho thấy tiềm năng của hướng tiếp cận này khi được huấn luyện ở quy mô đầy đủ.

## 2.2 Thiết kế các pipeline học máy

### 2.2.1 Tập dữ liệu EMBER2024

EMBER2024 [17] là tập dữ liệu benchmark quy mô lớn được thu thập từ tháng 9 năm 2023 đến tháng 12 năm 2024, bao gồm hơn 3,2 triệu file thuộc sáu định dạng khác nhau (Win32, Win64, .NET, APK, ELF và PDF). Vector đặc trưng sử dụng định dạng **EMBERv3** với 2.568 chiều, là phiên bản mở rộng so với EMBERv2, bổ sung thêm thông tin Rich header (dấu vết trình biên dịch), data directories, và nhóm đặc trưng tổng hợp các cảnh báo phát sinh trong quá trình phân tích file PE (PE file warnings). Các đặc trưng dạng danh mục hoặc chuỗi (tên section, tên hàm import, chuỗi văn bản) được biểu diễn dưới dạng vector hash kích thước cố định thông qua hashing trick (**FeatureHasher** của **sklearn**), đảm bảo chiều dài vector nhất quán.

Trong phạm vi thực nghiệm so sánh các pipeline, do giới hạn tài nguyên tính toán và thời gian chạy Capa, đề tài sử dụng một tập con gồm **450.000 mẫu huấn luyện** và **50.000 mẫu kiểm tra**, được lấy mẫu từ tập dữ liệu gốc với tỷ lệ malware/benign cân bằng và theo thứ tự thời gian: các mẫu cũ hơn dùng để huấn luyện (train), các mẫu mới hơn dùng để kiểm tra (test). Cách chia này mô phỏng sát điều kiện triển khai thực tế, trong đó mô hình phải tổng quát hóa sang các mẫu mã độc mới chưa từng xuất hiện trong dữ liệu huấn luyện.

Dưới đây trình bày chi tiết phương pháp triển khai của từng pipeline.

### 2.2.2 Pipeline A: EMBER2024 thuần

Pipeline A là baseline của thực nghiệm, sử dụng trực tiếp vector đặc trưng EMBERv3 (2.568 chiều) làm đầu vào cho LightGBM mà không bổ sung bất kỳ đặc trưng nào khác. Pipeline này phản ánh cách tiếp cận phổ biến nhất trong các nghiên cứu sử dụng tập dữ liệu EMBER, đồng thời là nền tảng để đánh giá mức độ đóng góp của đặc trưng Capa ở các pipeline sau. Việc huấn luyện lại pipeline A trên cùng tập con 450K mẫu - thay vì dùng mô hình pretrained gốc - nhằm đảm bảo việc so sánh các pipeline là công bằng: tất cả đều xuất phát từ cùng một điều kiện dữ liệu và cấu hình huấn luyện.

### 2.2.3 Pipeline B: EMBER2024 kết hợp Capa multi-hot

Pipeline B bổ sung đặc trưng năng lực (capability) từ Capa vào vector EMBER2024. Với mỗi file PE, Capa trả về kết quả phân tích rất chi tiết; song trong phạm vi đề tài, tác giả đã tiến hành cô đọng kết quả phân tích trên về cấu trúc gồm ba thành phần: danh sách **caps** (các capability được nhận diện kèm namespace phân cấp), danh sách **ttps** (chiến thuật và kỹ thuật theo MITRE ATT&CK), và danh sách **mbc** (hành vi theo Malware Behavior Catalog). Ví dụ, một file PE sử dụng kỹ thuật packing có thể được Capa nhận diện với capability **packed with upack** thuộc namespace **anti-analysis/packer/upack**, ánh xạ sang tactic **DEFENSE EVASION** trong ATT&CK và behavior **Software Packing** trong MBC. Quá trình cô đọng này được gọi là *remapping*.

```
1 {
2   "caps": [
3     {
4       "Capability": "reference analysis tools strings",
5       "Namespace": "anti-analysis",
6       "Addrs": []
7     },
8     {
9       "Capability": "packed with upack",
10      "Namespace": "anti-analysis/packer/upack",
11      "Addrs": []
12    }
13   ]
14 }
```

```

12     },
13     {
14         "Capability": "contain an embedded PE file",
15         "Namespace": "executable/subfile/pe",
16         "Addrs": []
17     },
18     {
19         "Capability": "contains PDB path",
20         "Namespace": "executable/pe/pdb",
21         "Addrs": []
22     },
23     {
24         "Capability": "(internal) packer file limitation",
25         "Namespace": "internal/limitation/static",
26         "Addrs": []
27     }
28 ],
29 "ttps": [
30     {
31         "Tactic": "DEFENSE EVASION",
32         "Technique": "Obfuscated Files or Information :: Software Packing"
33     }
34 ],
35 "mbc": [
36     {
37         "Objective": "DISCOVERY",
38         "Behavior": "Analysis Tool Discovery :: Process detection"
39     },
40     {
41         "Objective": "ANTI-STATIC ANALYSIS",
42         "Behavior": "Software Packing"
43     },
44     {
45         "Objective": "EXECUTION",
46         "Behavior": "Install Additional Program"
47     }
48 ]
49 }

```

Hình 2.1: Ví dụ output của Capa (remapped)

Trong pipeline B, tất cả các thành phần **caps**, **ttps** và **mbc** đều được sử dụng làm đặc trưng, và được gọi chung (trong khuôn khổ khóa luận này) là các capability. Từ điển capability toàn cục được xây dựng từ tập huấn luyện gồm **1.250 capabilities** khác nhau. Với mỗi file PE, danh sách capability được mã hóa thành vector nhị phân dạng **multi-hot**: mỗi chiều tương ứng với một capability trong từ điển, nhận giá trị 1 nếu file có capability đó và 0 nếu capability đó không hiện hữu trong file. Vector Capa multi-hot (1.250 chiều) sau đó được ghép nối với vector EMBER2024 (2.568 chiều) tạo thành vector đầu vào tổng hợp 3.818 chiều cho LightGBM.

Nhược điểm của cách mã hóa này là vector Capa multi-hot **rất thưa (sparse)**: với nhiều file PE, chỉ một phần nhỏ trong 1.250 capability được kích hoạt, dẫn đến phần lớn các chiều nhận giá trị 0. Điều này làm tăng chi phí tính toán và nguy cơ gây nhiễu cho mô hình. Pipeline C được thiết kế để khắc phục hạn chế này.

#### 2.2.4 Pipeline C: EMBER2024 kết hợp Capa TruncatedSVD

Pipeline C sử dụng lại vector Capa multi-hot của pipeline B, sau đó áp dụng kỹ thuật giảm chiều **TruncatedSVD** lên vector này trước khi ghép nối với vector EMBER2024 thuần để đưa vào mô hình LightGBM. TruncatedSVD phân tích ma trận đặc trưng Capa của toàn bộ tập huấn luyện, giữ lại  $k$  giá trị kỳ dị lớn nhất - tương ứng với các tổ hợp capability có tính phân biệt cao nhất giữa các mẫu - và chiếu xuống không gian  $k$  chiều. Ma trận chiếu được fit trên tập huấn luyện và áp dụng cố định lên tập kiểm tra, tránh data leakage.

Để xác định số chiều nén (giảm) phù hợp, thực nghiệm được thực hiện với ba giá trị: **64, 128 và 256 chiều**. Vector nén sau đó được ghép nối với vector EMBER2024 2.568 chiều, tạo thành vector đầu vào tổng hợp có số chiều lần lượt là 2.632, 2.696 và 2.824.

#### Lý do lựa chọn kỹ thuật giảm chiều TruncatedSVD

Vector Capa multi-hot là ma trận **rất thưa (sparse)**: với mỗi file PE, chỉ một phần nhỏ trong 1.250 capability được kích hoạt, dẫn đến phần lớn các chiều nhận giá trị 0. Đặc điểm này đặt ra yêu cầu cụ thể đối với kỹ thuật giảm chiều: cần hoạt động hiệu quả trên dữ liệu thưa và không làm mất đi ngữ nghĩa của giá trị 0.

PCA (Principal Component Analysis) là một trong các kỹ thuật giảm chiều được



sử dụng phổ biến nhất. PCA tìm các hướng có phương sai lớn nhất trong dữ liệu bằng cách phân tích ma trận hiệp phương sai  $\mathbf{C} = \frac{1}{n}\mathbf{X}^\top\mathbf{X}$ , trong đó  $\mathbf{X}$  là ma trận dữ liệu *đã được mean-center* - tức là đã trừ đi giá trị trung bình của từng chiều. Theo định nghĩa thống kê chuẩn, mean-centering là bước cần thiết để PCA phân tích đúng phương sai của dữ liệu. Tuy nhiên, với dữ liệu Capa multi-hot, thao tác này gây ra hai vấn đề. Thứ nhất, mean-centering phá vỡ cấu trúc thưa: sau khi trừ giá trị trung bình, các số 0 trở thành số âm và ma trận trở thành dense, làm tăng đáng kể chi phí bộ nhớ và tính toán. Thứ hai, và quan trọng hơn, giá trị 0 của một feature trong vector Capa multi-hot mang ý nghĩa xác định: file PE *không có* capability tương ứng với feature đó. Khi mean-centering biến các số 0 thành số âm, PCA diễn giải sự *vắng mặt* của một capability như một tín hiệu âm có trọng số - trong khi đó chỉ là trạng thái mặc định của nhiều file PE. Hệ quả là các thành phần chính bị nhiễu bởi việc dịch chuyển này, thay vì phản ánh các tổ hợp capability thực sự có tính phân biệt.

Để khắc phục hiện tượng trên, ta có thể bỏ qua bước mean-centering. Tuy nhiên, cần lưu ý rằng về mặt toán học, khi áp dụng PCA, việc bỏ qua bước mean-centering và chỉ giữ  $k$  hướng có phương sai lớn nhất (tương ứng với  $k$  giá trị kỳ dị lớn nhất) chính là áp dụng TruncatedSVD. Nói cách khác, nếu muốn dùng PCA mà tránh được vấn đề mean-centering trên dữ liệu sparse như đã nêu, kết quả thu được chính là TruncatedSVD. Do vậy, áp dụng TruncatedSVD trực tiếp là lựa chọn tự nhiên và đúng mục đích sử dụng cho bài toán này.

**Cơ sở của TruncatedSVD.** TruncatedSVD phân tích ma trận  $\mathbf{X}$  thành tích:

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top \quad (2.1)$$

trong đó  $\mathbf{\Sigma}$  là ma trận đường chéo chứa các giá trị kỳ dị  $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$  sắp xếp giảm dần. Mỗi  $\sigma_i$  phản ánh mức độ phương sai mà thành phần thứ  $i$  đóng góp vào toàn bộ dữ liệu:  $\sigma_i$  càng lớn thì thành phần đó càng mang nhiều thông tin phân biệt giữa các mẫu. TruncatedSVD chỉ giữ lại  $k$  giá trị kỳ dị lớn nhất, chiếu mỗi mẫu xuống không gian  $k$  chiều thông qua phép biến đổi  $\tilde{\mathbf{x}} = \mathbf{x}\mathbf{V}_k$ , hoạt động trực tiếp trên ma trận  $\mathbf{X}$  gốc mà không qua mean-centering.

Ma trận  $\mathbf{V}_k$  được fit trên tập huấn luyện và áp dụng cố định lên tập kiểm tra, tránh data leakage. Kết quả là mỗi file PE được biểu diễn bởi một vector  $k$  chiều phản ánh các tổ hợp capability có tính phân biệt cao nhất trong toàn bộ tập dữ liệu.

Như vậy, TruncatedSVD là lựa chọn phù hợp trong pipeline C, vừa đảm bảo hiệu quả tính toán trên tập dữ liệu quy mô lớn, vừa tương thích tự nhiên với đặc trưng Capa dạng thưa mà không làm mất đi tính chất thống kê quan trọng của dữ liệu.

### 2.2.5 Cấu hình huấn luyện

Tất cả các pipeline đều sử dụng mô hình LightGBM với cùng bộ siêu tham số, được lấy từ cấu hình gốc của dự án EMBER2024 [17]. Việc tái sử dụng cấu hình này có hai lý do: nhóm tác giả EMBER2024 đã tối ưu bộ siêu tham số cho bài toán phân loại mã độc PE trên tập dữ liệu này; và mục tiêu của thực nghiệm là so sánh hiệu quả của các *tổ hợp đặc trưng*, không phải tối ưu siêu tham số - việc giữ cố định siêu tham số đảm bảo rằng bất kỳ sự khác biệt nào quan sát được đều do đặc trưng đầu vào gây ra. Một số thiết lập đáng chú ý bao gồm `num_leaves = 64` và `lambda_l2 = 1.0` để kiểm soát độ phức tạp mô hình và tránh overfit; `bagging_fraction = 0.9` cùng `feature_fraction = 0.9` để tăng tính ngẫu nhiên trong quá trình xây dựng cây; và `is_unbalance = true` để xử lý mất cân bằng nhãn nếu có.

Quá trình huấn luyện áp dụng cơ chế dừng sớm `lgb.early_stopping(50)`: dừng huấn luyện nếu điểm ROC-AUC trên tập validation không cải thiện trong 50 vòng (rounds) liên tiếp. Cơ chế này vừa ngăn overfit, vừa tiết kiệm thời gian huấn luyện khi mô hình đã hội tụ. Số vòng tối đa là 500.

## 2.3 Đánh giá và so sánh các pipeline

### 2.3.1 Lựa chọn độ đo đánh giá

Trong bài toán phát hiện mã độc tại môi trường thực tế, False Positive Rate (FPR) - tỷ lệ dương tính giả (tỷ lệ file lành tính bị phân loại nhầm thành mã độc) - là chỉ số đặc biệt quan trọng. Một hệ thống cảnh báo nhầm quá nhiều sẽ mất đi tính khả dụng và gây mất niềm tin của người dùng, đặc biệt trong môi trường doanh nghiệp nơi hàng nghìn file được thực thi mỗi ngày. Do đó, đề tài sử dụng **Partial AUC tại FPR  $\leq 0,5\%$**  làm độ đo chính - chỉ số này đo khả năng phát hiện mã độc của mô hình trong vùng FPR cực thấp, phản ánh sát nhất yêu cầu vận hành thực tế. **Full AUC** được sử dụng như độ đo phụ để đánh giá hiệu quả tổng thể trên toàn bộ tất cả các ngưỡng phân loại.

### 2.3.2 Kết quả thực nghiệm

Bảng 2.1 trình bày ROC-AUC trên tập validation theo các mốc iteration và kết quả cuối cùng trên tập kiểm tra của 05 cấu hình được thử nghiệm.

Bảng 2.1: Kết quả so sánh các pipeline theo ROC-AUC (validation) và AUC (test set).

| Iteration                                   | Pipeline A<br>(EMBER2024) | Pipeline B<br>(+Capa multi-hot) | Pipeline C<br>(+Capa SVD-64) | Pipeline C<br>(+Capa SVD-128) | Pipeline C<br>(+Capa SVD-256) |
|---------------------------------------------|---------------------------|---------------------------------|------------------------------|-------------------------------|-------------------------------|
| 100                                         | 0.991358                  | 0.991224                        | 0.991288                     | 0.991317                      | 0.991116                      |
| 200                                         | 0.992569                  | 0.992687                        | 0.992706                     | 0.992782                      | 0.992563                      |
| 300                                         | 0.992794                  | 0.993042                        | 0.992997                     | 0.993162                      | 0.992987                      |
| 400                                         | — <sup>(1)</sup>          | 0.993130                        | 0.993082                     | 0.993283                      | 0.993159                      |
| 500                                         | — <sup>(1)</sup>          | 0.993251                        | 0.993194                     | — <sup>(2)</sup>              | 0.993282                      |
| <i>Kết quả trên tập kiểm tra (test set)</i> |                           |                                 |                              |                               |                               |
| Partial AUC (FPR $\leq$ 0,5%)               | 0.8816                    | 0.8843                          | 0.8830                       | 0.8838                        | <b>0.8850</b>                 |
| Full AUC                                    | 0.9922                    | <b>0.9927</b>                   | <b>0.9927</b>                | 0.9925                        | 0.9926                        |

<sup>(1)</sup> Dừng sớm (early stopping) tại iteration 251, ROC-AUC validation = 0.992813.

<sup>(2)</sup> Dừng sớm (early stopping) tại iteration 446, ROC-AUC validation = 0.993314.

### 2.3.3 Nhận xét kết quả

Từ kết quả trong Bảng 2.1, có thể rút ra một số nhận xét sau:

**Đặc trưng Capa có đóng góp thực chất.** Tất cả các pipeline có bổ sung đặc trưng Capa đều vượt baseline pipeline A trên Partial AUC - độ đo chính của thực nghiệm. Điều này cho thấy thông tin về năng lực mang lại giá trị phân biệt bổ sung mà đặc trưng EMBER2024 thuần túy chưa phản ánh đủ. Đáng chú ý, ở các iteration đầu (100–20 vòng), pipeline A và các pipeline Capa có kết quả xấp xỉ nhau; sự khác biệt chỉ bộc lộ rõ từ iteration 200 trở đi, khi mô hình bắt đầu khai thác sâu hơn các mối quan hệ trong đặc trưng.

**Pipeline A hội tụ sớm nhất.** Baseline dừng sớm tại iteration 251, trong khi các pipeline có Capa tiếp tục cải thiện đến iteration 400–500. Điều này phản ánh rằng việc bổ sung đặc trưng Capa làm tăng độ phức tạp của không gian đặc trưng, cho phép mô hình tiếp tục học thêm thông tin hữu ích trong các vòng huấn luyện tiếp theo.

**TruncatedSVD-256 là lựa chọn tốt nhất trong nhóm pipeline C.** Trong ba thử nghiệm giảm chiều bằng TruncatedSVD được thử nghiệm, SVD-256 đạt

Partial AUC cao nhất (0.8850) trên tập kiểm tra, vượt SVD-64 (0.8830) và SVD-128 (0.8838). Kết quả này cho thấy việc giữ lại nhiều chiều hơn giúp bảo toàn thông tin năng lực tốt hơn, đặc biệt trong vùng FPR thấp. SVD-128 dừng sớm nhất trong nhóm (iteration 446) và đạt Full AUC validation cao nhất (0.993314), nhưng lại không dẫn đến Partial AUC tốt nhất trên tập kiểm tra - điều này gợi ý rằng hội tụ nhanh không đồng nghĩa với hiệu năng tốt trong vùng FPR thấp.

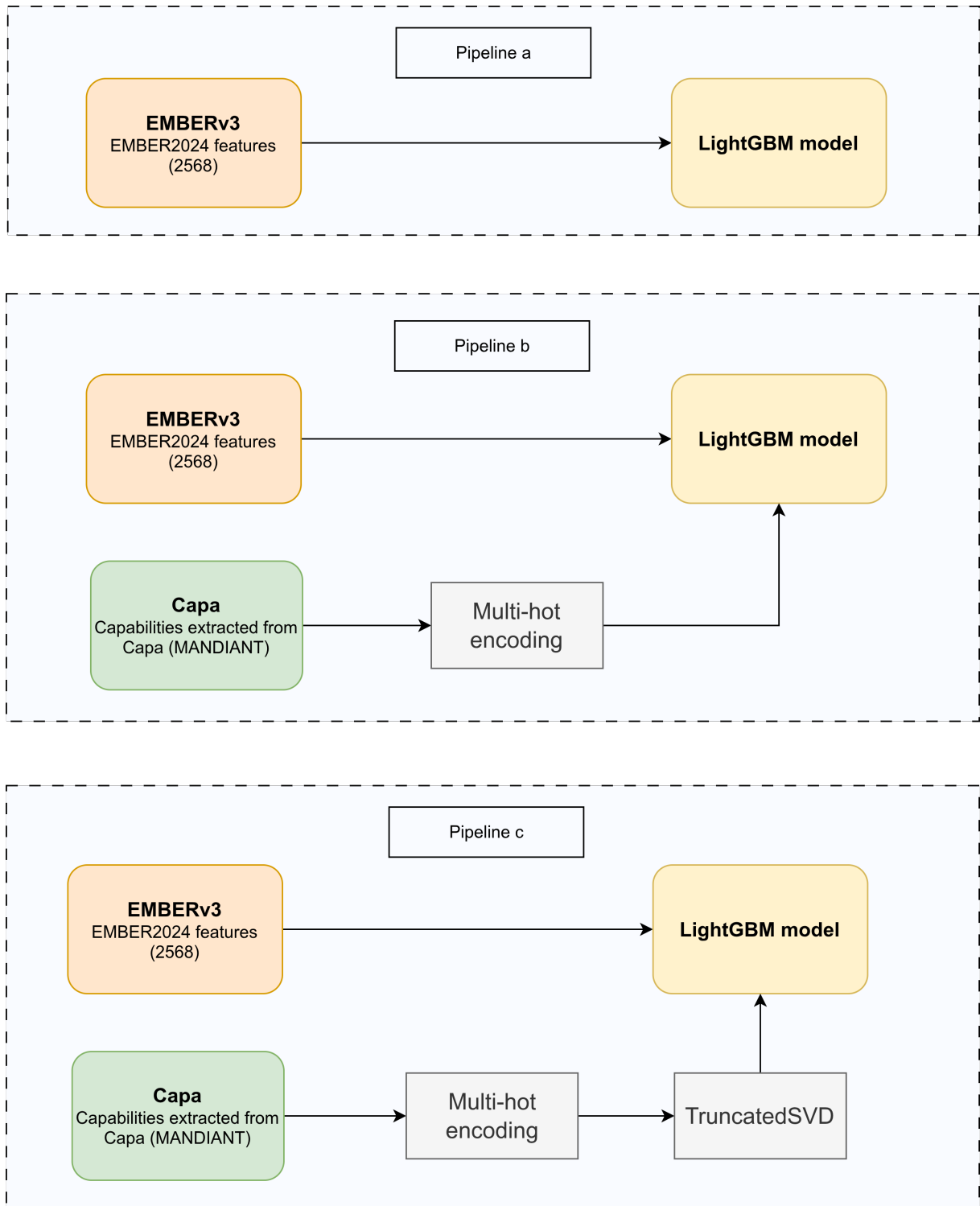
**So sánh pipeline B và pipeline C.** Pipeline B (Capa multi-hot) đạt Full AUC bằng với SVD-64 (0.9927), nhưng Partial AUC (0.8843) thấp hơn SVD-256 (0.8850). Điều này cho thấy TruncatedSVD không chỉ giúp giảm chi phí tính toán mà còn có tác dụng lọc nhiễu, loại bỏ các chiều capability ít có tính phân biệt và giữ lại các tổ hợp capability quan trọng nhất.

### 2.3.4 Thảo luận và lựa chọn pipeline triển khai

Dựa trên kết quả thực nghiệm, đề tài lựa chọn hai pipeline để đưa vào triển khai tại Chương 3, với tiêu chí cân bằng giữa hiệu quả phát hiện và yêu cầu vận hành:

**Pipeline A** được lựa chọn cho **dịch vụ antivirus offline tại endpoint**. Mặc dù Partial AUC thấp hơn các pipeline có Capa, pipeline A có ưu thế quyết định về tốc độ: trích xuất đặc trưng EMBER2024 thuần túy có thể thực hiện trong dưới một phân mười giây đối với file nhỏ, phù hợp với yêu cầu phân tích thời gian thực tại máy endpoint. Hơn nữa, khi triển khai tại endpoint, đề tài sử dụng mô hình LightGBM gốc được nhóm tác giả EMBER2024 cung cấp - đã được huấn luyện trên toàn bộ 2,6 triệu mẫu - thay vì mô hình huấn luyện trên tập con 450K mẫu, nhờ đó đảm bảo hiệu năng phân loại tốt nhất có thể cho dịch vụ offline.

**Pipeline C (SVD-256)** được lựa chọn cho **dịch vụ phân tích sâu online (deep-analysis-server)**. Trong ngữ cảnh này, thời gian phân tích có thể kéo dài hơn do người dùng chủ động yêu cầu phân tích một file cụ thể, và kết quả trả về cần mang nhiều thông tin ngữ nghĩa hơn - bao gồm danh sách capability được trích xuất và giải thích SHAP. Bên cạnh đó, pipeline C cũng đạt Partial AUC cao nhất (0.8850) trong số các pipeline được thử nghiệm.



Hình 2.1: Các pipeline học máy được sử dụng trong phát hiện mã độc PE.

# Chương 3

## Phát triển dịch vụ phát hiện mã độc trên máy endpoint và dịch vụ phân tích mã độc online

### 3.1 Phân tích yêu cầu hệ thống

#### 3.1.1 Yêu cầu chức năng

Hệ thống được thiết kế để cung cấp hai lớp bảo vệ bổ sung cho nhau: lớp phát hiện nhanh tại endpoint và lớp phân tích chuyên sâu trực tuyến (online).

**Tại máy endpoint**, hệ thống cần đáp ứng các yêu cầu chức năng sau:

- Tự động phát hiện và chặn thực thi file PE bị phân loại là mã độc ngay tại thời điểm tiến trình được tạo, mà không yêu cầu thao tác thủ công từ người dùng.
- Hiện thị thông báo (popup) ngay lập tức khi phát hiện và chặn một mã độc mới.
- Cho phép người dùng xem lịch sử các lần phát hiện, bao gồm đường dẫn file và điểm đánh giá mức độ nguy hiểm (severity score).
- Cho phép người dùng submit file đáng ngờ lên dịch vụ phân tích sâu trực tuyến để nhận kết quả chi tiết hơn.

**Tại dịch vụ phân tích sâu trực tuyến (deep-analysis-server)**, hệ thống cần đáp ứng các yêu cầu sau:

- Nhận file PE từ người dùng thông qua giao diện web hoặc REST API.
- Thực hiện phân tích đầy đủ theo pipeline c (EMBER2024 + Capa + TruncatedSVD-256 + LightGBM + SHAP) và trả về kết quả phân loại cùng giải thích chi tiết.

- Hiển thị danh sách từng đặc trưng và mức độ đóng góp (contribution) vào quyết định phân loại, sắp xếp theo thứ tự giảm dần; đồng thời tổng hợp theo nhóm đặc trưng.
- Lưu trữ kết quả phân tích theo hash của file, tránh phân tích lại các file đã từng được xử lý thành công.
- Cung cấp chức năng thử lại (Retry) đối với các file đã upload nhưng quá trình phân tích gặp lỗi ngoại lệ (exception), mà không yêu cầu người dùng upload lại file.

### 3.1.2 Yêu cầu phi chức năng

- **Hiệu năng tại endpoint:** Quá trình trích xuất đặc trưng và suy luận mô hình cần hoàn thành trong thời gian đủ ngắn để không gây trải nghiệm chờ đợi đáng chú ý cho người dùng. Đây là lý do module trích xuất đặc trưng được cài đặt bằng C++ thay vì Python (xem Mục 3.2.1).
- **Quyền hệ thống:** pmd-service và pmd-ui cần được chạy với quyền quản trị (administrator) để có thể giao tiếp với driver qua IOCTL. pmd-service tự cài đặt vào danh mục Windows service và được cấu hình khởi động cùng hệ điều hành.
- **Hạn chế triển khai driver:** Driver trong phạm vi đề tài không được ký bằng chứng thực số (codesigning) cho môi trường production, do đó việc nạp driver chỉ được thực hiện trên máy ảo thử nghiệm đã được provisioning phục vụ phát triển driver (thông qua WDK/Visual Studio, sử dụng tài khoản WDKRemoteUser và cơ chế test signing).

Nguyên nhân là trên các phiên bản Windows 10/11 64-bit hiện đại, đặc biệt khi Secure Boot được bật, kernel-mode driver thường phải được Microsoft xác thực chữ ký thông qua Hardware Dev Center. Quy trình này yêu cầu nhà phát triển sở hữu tài khoản Microsoft Partner Center đã được xác minh cùng chứng thư số Extended Validation (EV Code Signing Certificate) do tổ chức chứng thực (Certificate Authority) được Microsoft tin cậy cấp phát. Các yêu cầu này phát sinh chi phí đáng kể và nằm ngoài phạm vi của đề tài.

Hạn chế trên được chấp nhận do mục tiêu chính của hệ thống là nghiên cứu kiến trúc và kiểm chứng tính khả thi của giải pháp, thay vì triển khai trên môi trường thực tế với người dùng thật.

- **Tính sẵn sàng của dịch vụ online:** Deep-analysis-server được đóng gói bằng Docker, đảm bảo tính nhất quán về môi trường thực thi và đơn giản hóa quá trình triển khai.

### 3.1.3 Tổng quan các thành phần hệ thống

Hệ thống gồm năm thành phần chính, được mô tả tổng quan trong Bảng 3.1 và Hình 3.1.

Bảng 3.1: Các thành phần chính của hệ thống

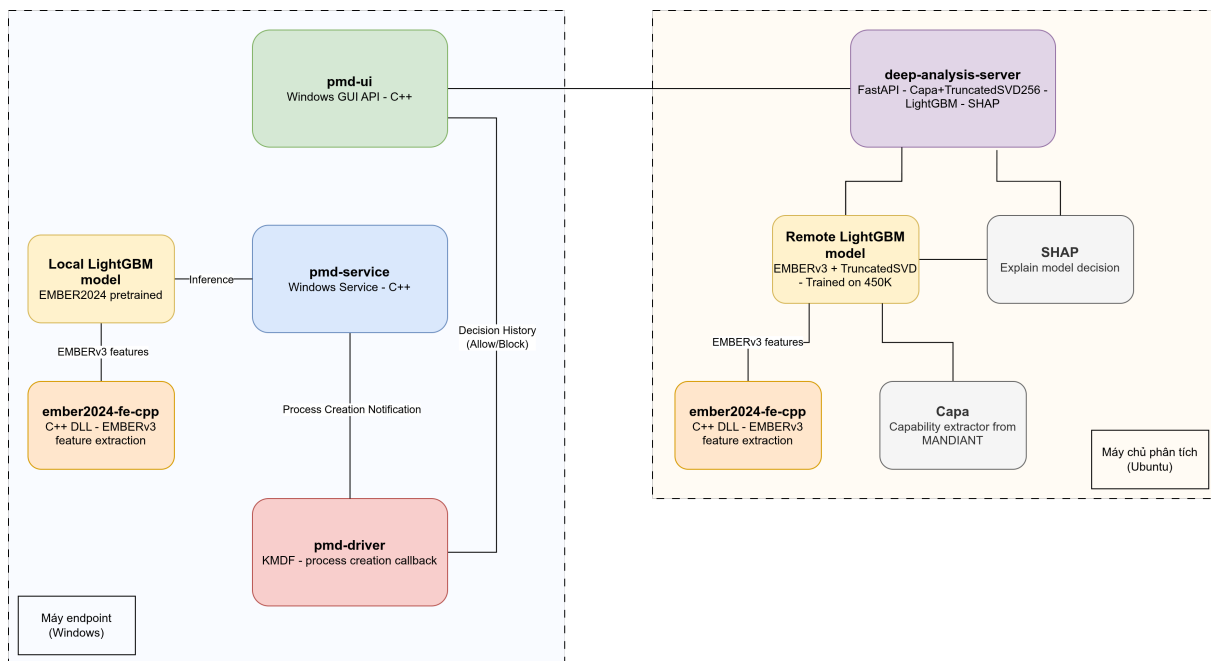
| Module               | Ngôn ngữ        | Chức năng chính                                                  |
|----------------------|-----------------|------------------------------------------------------------------|
| ember2024-fe-cpp     | C++, CMake      | Trích xuất vector đặc trưng EMBERv3 từ file PE                   |
| pmd-driver           | C, KMDF         | Phát hiện tiến trình mới, giao tiếp với pmd-service qua IOCTL    |
| pmd-service          | C++             | Windows service nhận scan task, chạy mô hình ML, trả verdict     |
| pmd-ui               | C++, Win API    | Hiển thị lịch sử phát hiện, thông báo mã độc, submit file online |
| deep-analysis-server | Python, FastAPI | Phân tích sâu với Capa + SVD-256 + SHAP, giao diện web           |

## 3.2 Thiết kế hệ thống chi tiết

### 3.2.1 Module trích xuất đặc trưng (ember2024-fe-cpp)

Module ember2024-fe-cpp cài đặt lại pipeline trích xuất đặc trưng EMBERv3 bằng C++, cho phép tích hợp trực tiếp vào các thành phần hệ thống mà không cần phụ thuộc vào môi trường Python. Module được quản lý bằng CMake và được biên dịch (compile) thành một thư viện liên kết động (DLL) để có thể được sử dụng bởi





Hình 3.1: Kiến trúc tổng thể của hệ thống phát hiện mã độc trên máy endpoint.

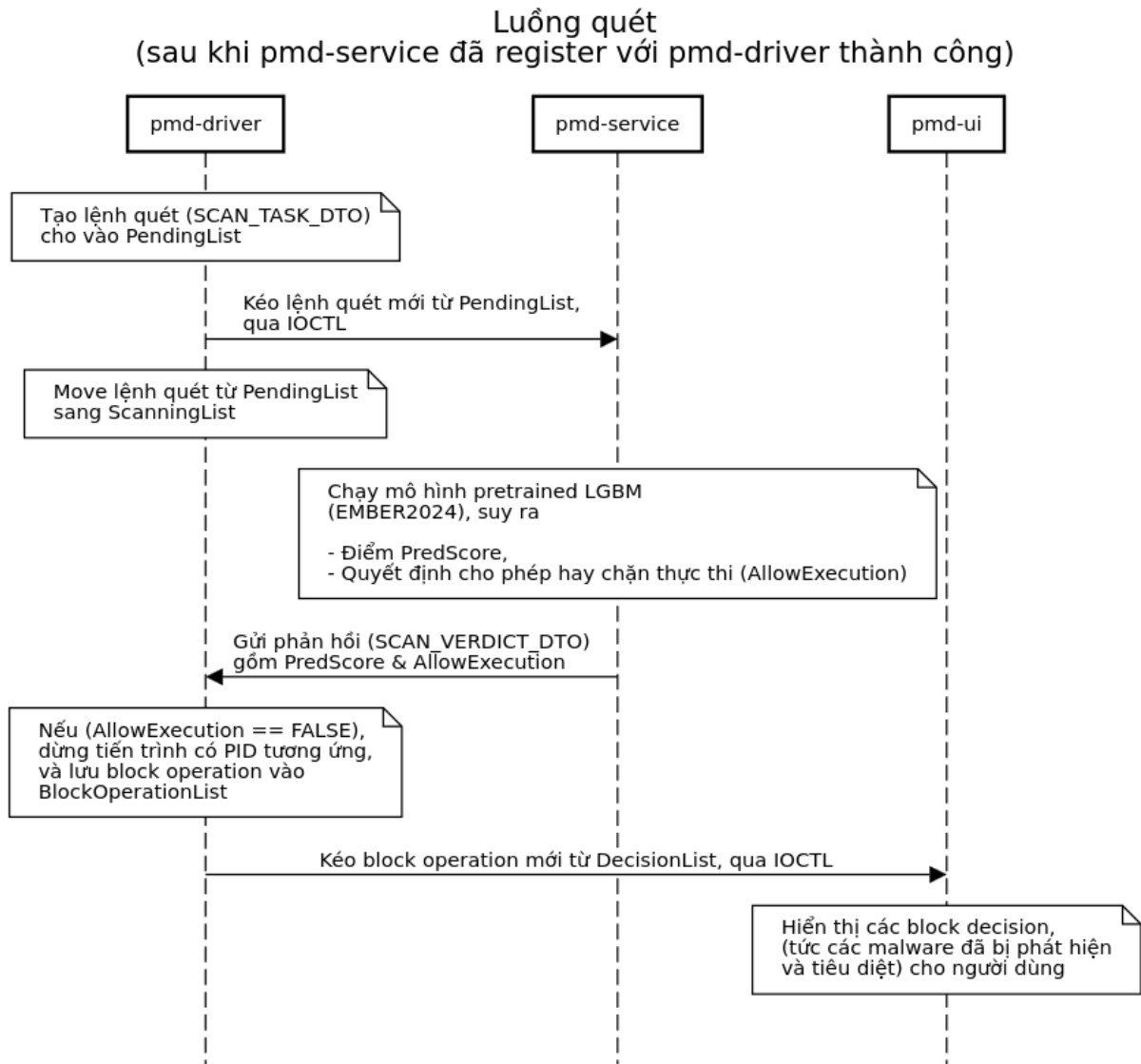
pmd-service (qua lời gọi hàm LoadLibrary) và deep-analysis-server (qua Python FFI).

Lý do cốt lõi cho quyết định cài đặt lại bằng C++ là yêu cầu hiệu năng tại endpoint: trích xuất đặc trưng cần hoàn thành trong thời gian đủ ngắn để không block tiến trình đang chờ kết quả quét từ pmd-service. Bảng 3.2 trình bày kết quả so sánh hiệu năng giữa cài đặt Python gốc và cài đặt C++ trên các file PE có kích thước khác nhau.

Bảng 3.2: So sánh hiệu năng giữa hai bộ trích xuất đặc trưng EMBER2024 - Python và C++

| Kích thước file PE | Python (giây) | C++ (giây) | Mức tăng tốc |
|--------------------|---------------|------------|--------------|
| 1,5 MB             | 0,657         | 0,053      | 1240%        |
| 4,5 MB             | 1,519         | 0,114      | 1332%        |
| 28,7 MB            | 13,904        | 0,689      | 2018%        |

Kết quả cho thấy cài đặt C++ nhanh hơn cài đặt Python từ 12 đến 20 lần tùy theo kích thước file, với mức tăng càng lớn khi file càng lớn. Điều này đặc biệt quan trọng với các file PE lớn - vốn phổ biến trong các bộ cài đặt phần mềm - nơi cài đặt Python mất gần 14 giây trong khi cài đặt C++ chỉ cần dưới 0.7 giây.



Hình 3.2: Sequence diagram thể hiện luồng quét file PE tại máy endpoint

### 3.2.2 Module driver (pmd-driver)

pmd-driver được viết bằng C, dựa trên KMDF (Kernel-Mode Driver Framework). Driver sử dụng cơ chế **process creation callback** để nhận thông báo từ hệ điều hành mỗi khi có một tiến trình mới được tạo, từ đó phát hiện các lần thực thi file PE.

**Cơ chế hoạt động.** Khi được nạp vào hệ điều hành, driver ở trạng thái nghỉ và chưa theo dõi bất kỳ tiến trình nào. Trạng thái này kéo dài cho đến khi pmd-service gửi một IOCTL đặc biệt để đăng ký (register), thông báo cho driver rằng đã có một scanner sẵn sàng nhận và xử lý các yêu cầu quét. Sau khi đăng ký thành công, mỗi khi phát hiện một tiến trình mới, driver tạo một lệnh quét (scan task) dưới

dạng `SCAN_TASK_DTO` với các trường thông tin sau:

- `PID`: định danh của tiến trình cần quét.
- `FileID` (128-bit): định danh duy nhất của file PE trên hệ thống file.
- `VolumeSerialNumber`: serial number của ổ đĩa chứa file PE.
- `Timestamp`: thời điểm tạo lệnh quét.

Lệnh quét được gửi lên `pmd-service` qua `IOCTL`. Driver sau đó chờ phản hồi dưới dạng `SCAN_VERDICT_DTO` từ `pmd-service`, bao gồm:

- `PID`, `FileID`, `VolumeSerialNumber`: định danh tiến trình và file đã quét.
- `PredScore`: xác suất file là mã độc (kiểu `double`, trong khoảng `[0.0, 1.0]`).
- `AllowExecution`: quyết định cho phép hay chặn thực thi (kiểu `BOOLEAN`).

Dựa trên `AllowExecution`, driver thực hiện một trong hai hành động: nếu `TRUE`, tiến trình được phép chạy bình thường; nếu `FALSE`, driver lập tức dừng (terminate) tiến trình có `PID` tương ứng và lưu lại quyết định chặn dưới dạng struct `BLOCK_OPERATION_DTO` để `pmd-ui` có thể đọc lịch sử.

### 3.2.3 Module dịch vụ nền (`pmd-service`)

`pmd-service` là một Windows service viết bằng C++, đóng vai trò là bộ quét (scanner) trung tâm tại endpoint. Service cần được cài đặt với flag `-install` dưới quyền administrator, sau đó được đăng ký vào danh mục Windows service với cấu hình tự khởi động cùng hệ điều hành.

**Quy trình khởi động và đăng ký với driver.** Khi service khởi động, bước đầu tiên là gửi một `IOCTL` đăng ký đến driver để thông báo rằng scanner đã sẵn sàng. Chỉ sau khi đăng ký thành công, driver mới bắt đầu gửi các scan task xuống service.

**Luồng xử lý một scan task.** Khi nhận được `SCAN_TASK_DTO` từ driver, `pmd-service` thực hiện các bước sau:

1. Định vị file PE trên hệ thống file dựa trên `FileID` và `VolumeSerialNumber`.

2. Gọi `ember2024-fe-cpp` để trích xuất vector đặc trưng EMBERv3 2.568 chiều từ file PE.
3. Đưa vector đặc trưng vào mô hình LightGBM pretrained (EMBER2024) để nhận điểm `PredScore`.
4. So sánh `PredScore` với ngưỡng phát hiện đã định trước để xác định `AllowExecution`.
5. Gửi `SCAN_VERDICT_DTO` phản hồi về driver qua IOCTL.

### 3.2.4 Module giao diện người dùng (pmd-ui)

`pmd-ui` là ứng dụng Windows với giao diện đồ họa (GUI) đơn giản, viết bằng C++ và chỉ sử dụng Windows API. Ứng dụng cần được chạy với quyền administrator để có thể giao tiếp với driver qua IOCTL.

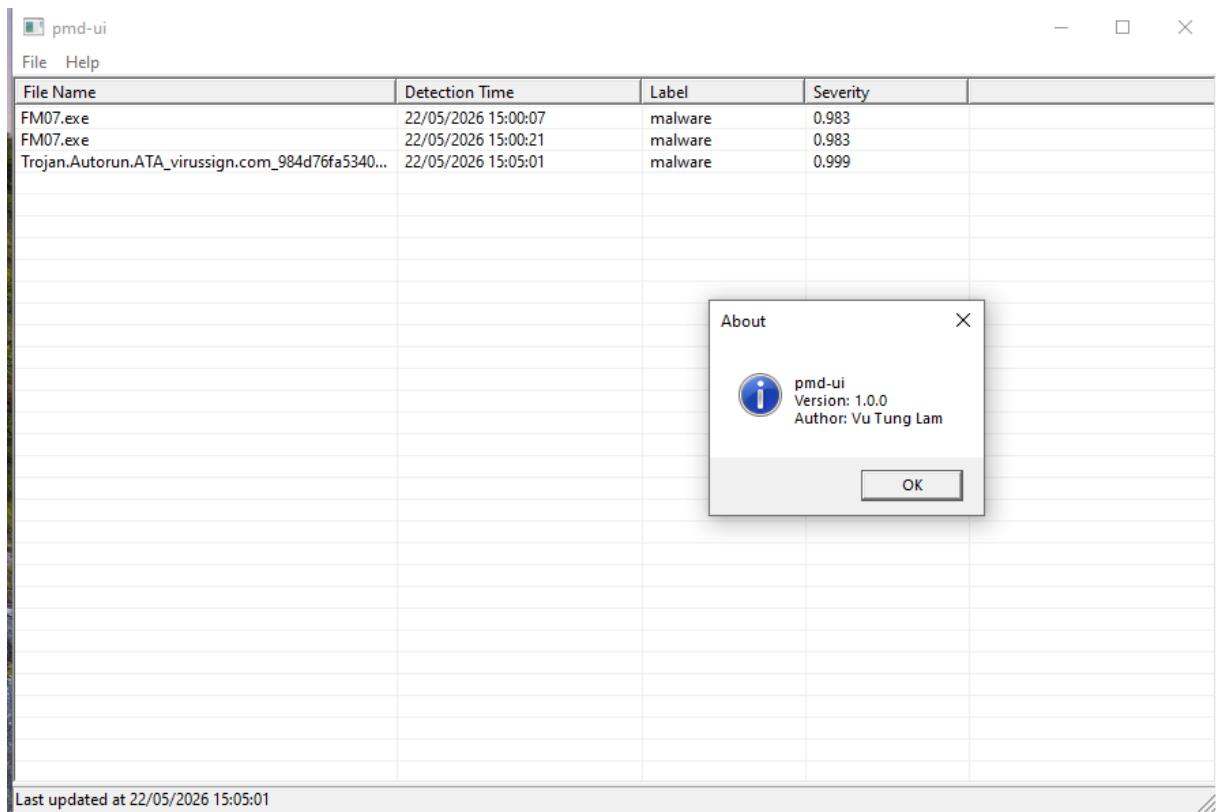
`pmd-ui` cung cấp các chức năng sau:

- **Hiển thị lịch sử phát hiện:** đọc danh sách các `BLOCK_OPERATION_DTO` từ driver, hiển thị thông tin từng lần chặn bao gồm đường dẫn file và điểm `PredScore`.
- **Thông báo thời gian thực:** hiển thị popup ngay khi driver phát hiện và chặn một mã độc mới.
- **Submit phân tích sâu:** từ màn hình chi tiết của một lần phát hiện, người dùng có thể submit file lên `deep-analysis-server` để nhận phân tích đầy đủ với giải thích SHAP.

### 3.2.5 Module phân tích sâu (deep-analysis-server)

`deep-analysis-server` là dịch vụ REST API viết bằng Python với framework FastAPI, được đóng gói bằng Docker. Dịch vụ này triển khai pipeline `c` (EMBER2024 + Capa + TruncatedSVD-256 + LightGBM + SHAP) và cung cấp giao diện web để người dùng tương tác.

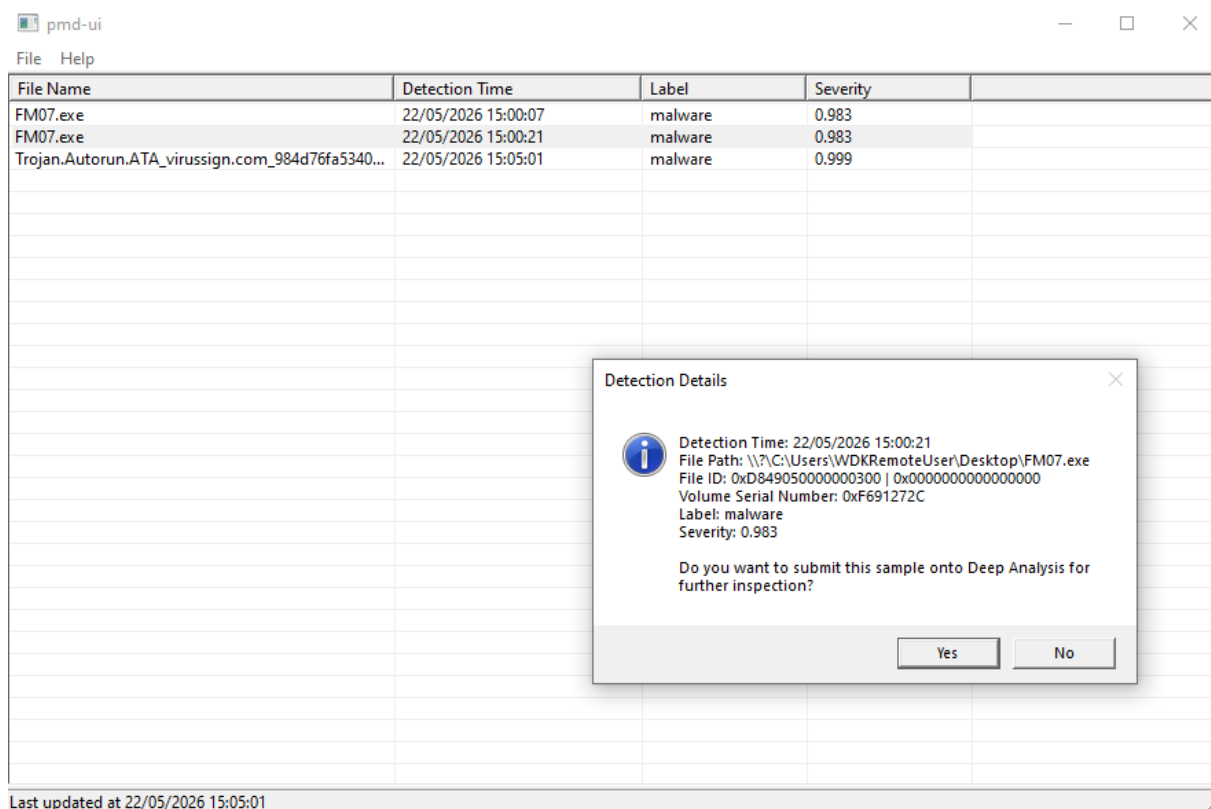
**Luồng xử lý phân tích.** Khi nhận được một file PE (qua giao diện web hoặc API), dịch vụ thực hiện các bước sau:



Hình 3.3: Giao diện chính của pmd-ui hiển thị lịch sử các lần phát hiện mã độc.

1. Tính hash của file và kiểm tra cơ sở dữ liệu lưu trữ kết quả. Nếu file đã được phân tích thành công trước đó, trả về kết quả đã lưu mà không phân tích lại.
2. Gọi ember2024-fe-cpp để trích xuất vector đặc trưng EMBERv3.
3. Chạy Capa trên file PE để lấy danh sách capability, sau đó mã hóa one-hot và giảm chiều xuống 256 chiều bằng TruncatedSVD đã được huấn luyện từ trước.
4. Ghép nối vector EMBER2024 và vector Capa đã giảm chiều, đưa vào mô hình LightGBM để nhận **PredScore** (xác suất file là mã độc - nhận giá trị từ 0.0 đến 1.0).
5. Tính toán SHAP values cho kết quả dự đoán, tổng hợp theo từng đặc trưng và theo nhóm đặc trưng.
6. Lưu kết quả vào cơ sở dữ liệu theo hash file.

Nếu bất kỳ bước nào phát sinh ngoại lệ (exception), kết quả được lưu với trạng

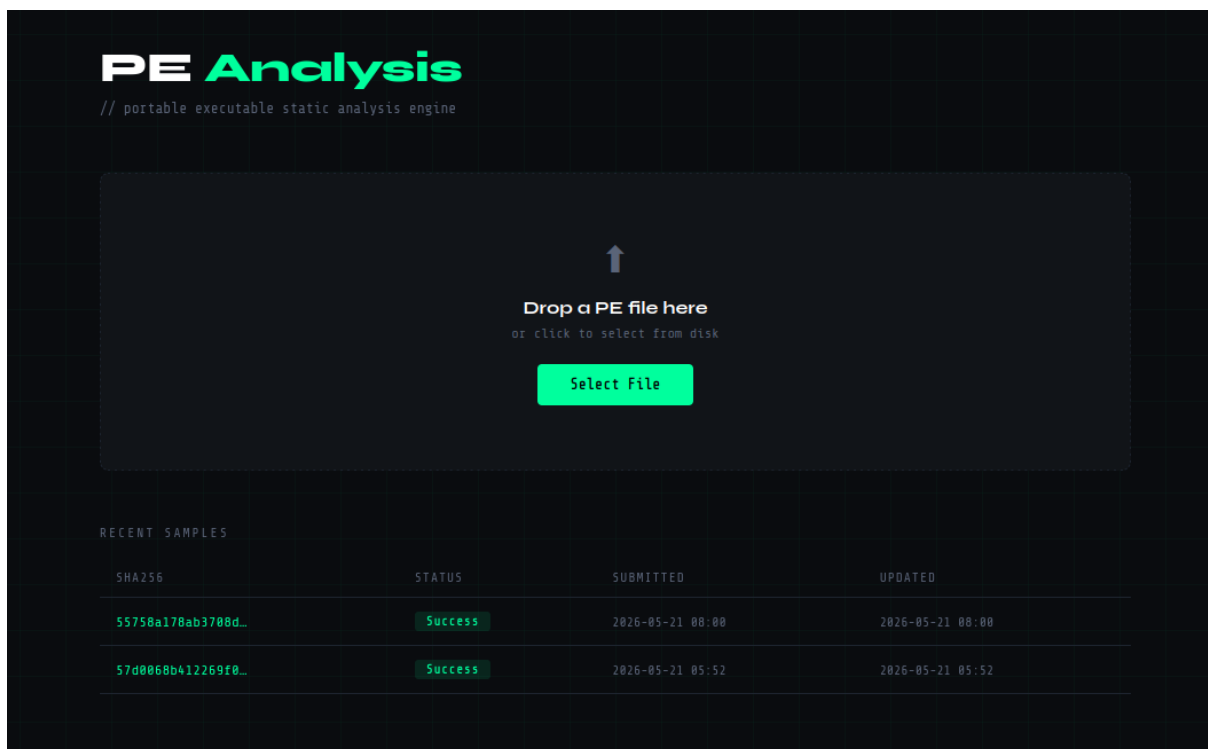


Hình 3.4: Giao diện của pmd-ui gợi ý gửi mã độc lên dịch vụ phân tích chuyên sâu (deep-analysis-server).

thái lỗi. Người dùng có thể kích hoạt lại phân tích bằng chức năng **Retry** mà không cần upload lại file (nếu quá trình upload file đã thành công trước đó).

**Kết quả phân tích trả về** bao gồm:

- Nhân phân loại (lành tính / mã độc) và điểm **PredScore**.
- Danh sách từng đặc trưng và giá trị SHAP contribution tương ứng, sắp xếp theo thứ tự đóng góp giảm dần.
- Danh sách các nhóm đặc trưng (PE header, section, import table, chuỗi văn bản, nội dung nhị phân, Capa... ) và tổng SHAP contribution của từng nhóm, sắp xếp theo thứ tự giảm dần.



Hình 3.5: Giao diện web của deep-analysis-server, cho phép người dùng upload file thủ công.

### 3.3 Cài đặt và kiểm thử

#### 3.3.1 Môi trường cài đặt

#### 3.3.2 Mô tả phần tự cài đặt và phần tận dụng

Bảng 3.4 mô tả phân tách giữa phần sinh viên tự cài đặt và phần tận dụng từ thư viện hoặc công cụ bên ngoài.

#### 3.3.3 Kịch bản kiểm thử end-to-end

Các kịch bản kiểm thử tập trung vào luồng hoạt động tổng thể của hệ thống, từ thời điểm file PE được thực thi đến khi người dùng nhận được kết quả phân tích.

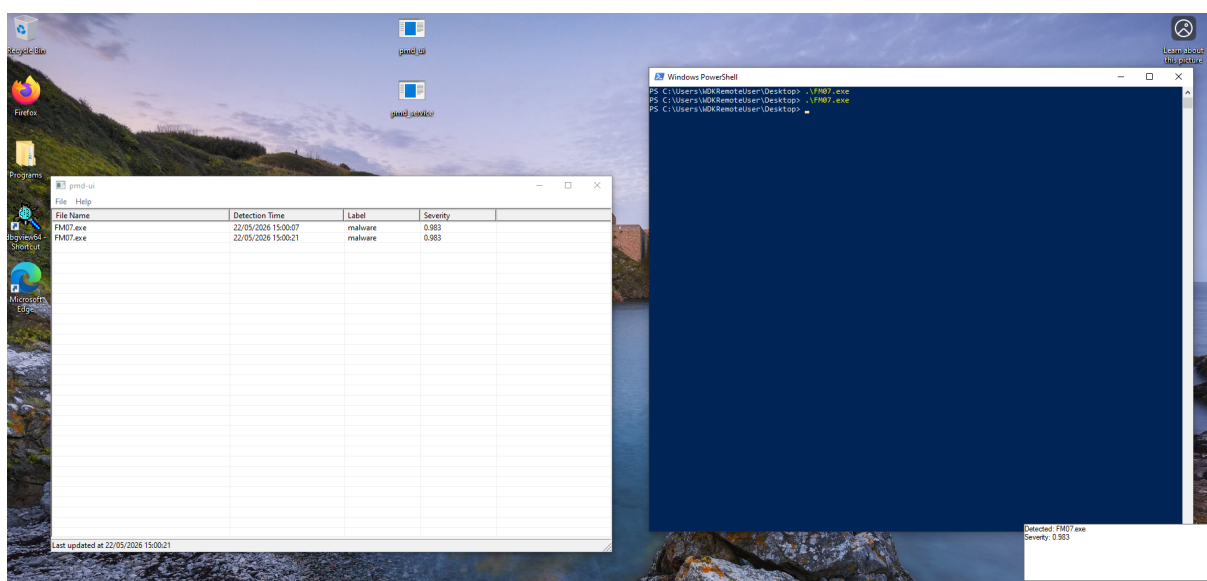
#### 3.3.4 Kết quả kiểm thử và đánh giá hiệu năng

**Kết quả kiểm thử.**

**Đánh giá hiệu năng trích xuất đặc trưng.** Bảng 3.2 (Mục 3.2.1) đã trình bày

Bảng 3.3: Môi trường cài đặt và kiểm thử

| Thành phần         | Cấu hình                                   |
|--------------------|--------------------------------------------|
| Hệ điều hành       | Windows 10 (đã tắt Windows Defender)       |
| Máy ảo driver      | Windows (WDKRemoteUser, Provisioning mode) |
| Ngôn ngữ / Runtime | C++ (MSVC 2022), Python 3.12, CMake        |
| Framework driver   | KMDF (Windows Driver Kit)                  |
| Docker             | Docker Engine on Ubuntu 22.04              |



Hình 3.6: Kết quả TC2: tiến trình bị terminate và popup thông báo trên pmd-ui.

kết quả so sánh hiệu năng giữa cài đặt Python và C++ của bộ trích xuất đặc trưng EMBER2024. Với mức tăng tốc từ 12 đến 20 lần, cài đặt C++ đảm bảo rằng tổng thời gian xử lý tại endpoint - bao gồm trích xuất đặc trưng và suy luận LightGBM - nằm trong giới hạn chấp nhận được đối với người dùng cuối, kể cả với các file PE có kích thước lớn.



Bảng 3.4: Phân tách phần tự cài đặt và phần tận dụng

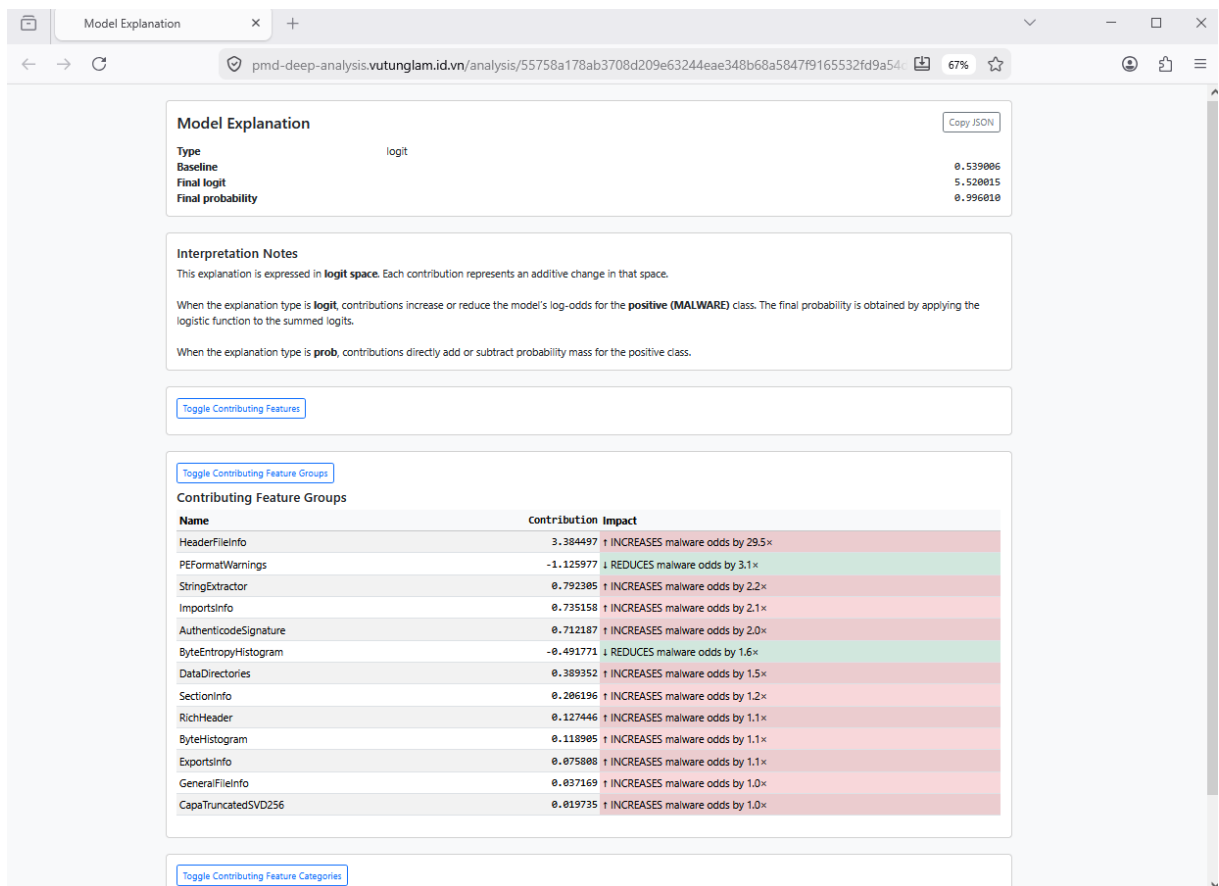
| Thành phần           | Loại       | Mô tả                                               |
|----------------------|------------|-----------------------------------------------------|
| ember2024-fe-cpp     | Tự cài đặt | Cài đặt lại pipeline EMBERv3 bằng C++               |
| pmd-driver           | Tự cài đặt | KMDF driver với process creation callback, IOCTL    |
| pmd-service          | Tự cài đặt | Windows service, tích hợp ember2024-fe-cpp          |
| pmd-ui               | Tự cài đặt | GUI Windows API, đọc IOCTL, submit file online      |
| FastAPI server       | Tự cài đặt | REST API, luồng phân tích, lưu kết quả theo hash    |
| Giao diện web        | Tự cài đặt | HTML/CSS/JS, hiển thị SHAP, Retry                   |
| LightGBM (EMBER2024) | Tận dụng   | Pretrained model do nhóm tác giả EMBER2024 cung cấp |
| Capa                 | Tận dụng   | Công cụ capability extraction (Mandiant)            |
| SHAP                 | Tận dụng   | Thư viện giải thích mô hình (xAI)                   |
| TruncatedSVD         | Tận dụng   | scikit-learn, fit trên tập huấn luyện 450K mẫu      |
| KMDF Framework       | Tận dụng   | Windows Driver Kit (Microsoft)                      |

Bảng 3.5: Kịch bản kiểm thử end-to-end

| TC  | Mô tả                      | Đầu vào                                     | Kết quả kỳ vọng                                                                     |
|-----|----------------------------|---------------------------------------------|-------------------------------------------------------------------------------------|
| TC1 | Thực thi file lành tính    | File PE đã biết là lành tính                | Tiến trình được phép chạy, không có thông báo                                       |
| TC2 | Thực thi file mã độc       | File PE mã độc đã biết                      | Tiến trình bị terminate, popup thông báo xuất hiện trên pmd-ui                      |
| TC3 | Xem chi tiết lần phát hiện | Chọn một mục trong lịch sử phát hiện        | Hiển thị đường dẫn file và severity score                                           |
| TC4 | Submit phân tích sâu       | Submit file từ TC2 lên deep-analysis-server | Nhận kết quả phân loại, danh sách feature contribution và nhóm feature contribution |
| TC5 | Phân tích file đã biết     | Upload lại file đã phân tích ở TC4          | Trả về kết quả đã lưu, không phân tích lại                                          |
| TC6 | Retry sau lỗi              | Kích hoạt Retry trên file có trạng thái lỗi | Hệ thống phân tích lại mà không yêu cầu upload lại file                             |

Bảng 3.6: Kết quả kiểm thử end-to-end

| TC  | Mô tả                   | Kết quả | Ghi chú                          |
|-----|-------------------------|---------|----------------------------------|
| TC1 | Thực thi file lành tính | Pass    | Hoạt động đúng như mong đợi      |
| TC2 | Thực thi file mã độc    | Pass    | Phát hiện và cảnh báo thành công |
| TC3 | Xem chi tiết phát hiện  | Pass    | Hiển thị đầy đủ thông tin        |
| TC4 | Submit phân tích sâu    | Pass    | Tác vụ được gửi thành công       |
| TC5 | Phân tích file đã biết  | Pass    | Trả về kết quả từ cache          |
| TC6 | Retry sau lỗi           | Pass    | Cho phép thực hiện lại tác vụ    |



Hình 3.7: Kết quả TC4: giao diện deep-analysis-server hiển thị kết quả phân loại và danh sách đóng góp đặc trưng.

# Kết luận và hướng phát triển

Khóa luận đã giải quyết hai bài toán có liên hệ mật thiết với nhau: nghiên cứu phương pháp học máy để phát hiện mã độc PE và triển khai kết quả nghiên cứu vào một hệ thống bảo vệ endpoint hoạt động được trên Windows.

Về bài toán nghiên cứu, khóa luận đã xây dựng và đánh giá thực nghiệm ba pipeline học máy trên tập dữ liệu EMBER2024, tập trung vào việc đánh giá đóng góp của kỹ thuật trích xuất năng lực mã độc (capability extraction) bằng công cụ Capa. Kết quả cho thấy tất cả các pipeline có bổ sung đặc trưng Capa đều vượt baseline EMBER2024 thuần túy trên Partial AUC ( $FPR \leq 0,5\%$ ). Pipeline kết hợp EMBER2024 với Capa đã giảm chiều bằng TruncatedSVD-256 đạt kết quả tốt nhất với Partial AUC = 0,8850 và Full AUC = 0,9926, cải thiện lần lượt 0,34% và 0,04% so với baseline (0,8816 và 0,9922). Kết quả này xác nhận rằng thông tin ngữ nghĩa hành vi từ Capa mang lại giá trị phân biệt bổ sung thực chất, đồng thời cho thấy TruncatedSVD vừa phù hợp với đặc trưng dạng thừa của Capa, vừa có tác dụng lọc nhiễu tốt hơn so với multi-hot encoding thuần túy.

Về bài toán công nghệ, khóa luận đã thiết kế và hiện thực hóa kiến trúc hệ thống bảo vệ endpoint gồm năm thành phần: module trích xuất đặc trưng C++ (ember2024-fe-cpp) đạt tốc độ nhanh hơn cài đặt Python gốc từ 12 đến 20 lần; kernel driver phát hiện tiến trình thực thi (pmd-driver); dịch vụ nền tích hợp mô hình học máy (pmd-service); giao diện người dùng với thông báo thời gian thực (pmd-ui); và dịch vụ phân tích sâu trực tuyến tích hợp Capa và SHAP (deep-analysis-server). Việc tích hợp SHAP theo nhóm đặc trưng giúp người dùng hiểu được lý do mô hình đưa ra quyết định phân loại, nâng cao tính minh bạch và khả năng ứng dụng thực tế của hệ thống.

Tuy nhiên, khóa luận vẫn còn một số hạn chế. Về phía nghiên cứu, thực nghiệm so sánh các pipeline được thực hiện trên tập con 450.000 mẫu thay vì toàn bộ tập EMBER2024, do giới hạn về thời gian chạy Capa. Đề tài cũng chỉ tập trung vào phân tích tĩnh và một mô hình học máy duy nhất (LightGBM), chưa so sánh với các kiến trúc học sâu. Về phía hệ thống, driver chưa được ký số nên chỉ có thể nạp qua chế độ Provisioning trên máy ảo phát triển (dev machine), không phù hợp với triển khai trên máy người dùng thực tế (song đây là giới hạn của phạm vi đề tài).

Ngoài ra, kernel module hiện chưa có các cơ chế chống can thiệp (anti-tampering) cần thiết cho môi trường triển khai thực tế.

Từ những kết quả và hạn chế trên, khóa luận đề xuất một số hướng phát triển tiếp theo. Trước hết, cần mở rộng thực nghiệm trên toàn bộ tập EMBER2024 và so sánh với các kiến trúc học sâu để có đánh giá toàn diện hơn về hiệu quả của capability extraction. Tiếp theo, mặc dù bổ sung đặc trưng Capa mang lại cải thiện có ý nghĩa, mức cải thiện còn khiêm tốn - điều này gợi ý rằng việc biểu diễn capability ở mức hành vi đơn lẻ chưa khai thác hết tiềm năng của thông tin ngữ nghĩa. Một hướng nghiên cứu đầy triển vọng là nâng mức trừu tượng từ capability lên *intent* - tức là suy luận ý đồ của mã độc (đánh cắp dữ liệu, duy trì/persistence, tránh phát hiện/evade detection...) từ sự kết hợp của nhiều capability, thay vì xử lý từng capability độc lập. Đây là một khoảng trống nghiên cứu (research gap) còn khá mở, trong đó các framework như MITRE ATT&CK có thể đóng vai trò nền tảng để xây dựng biểu diễn đặc trưng cấp cao hơn. Song song đó, tích hợp phân tích động theo hướng hybrid analysis - ví dụ thông qua knowledge distillation như đã trình bày trong Chương 1 - có thể nâng cao khả năng phát hiện các mẫu mã độc trốn tránh phân tích tĩnh. Về phía hệ thống, các hướng phát triển tiếp theo bao gồm cho phép cấu hình ngưỡng phân loại (threshold) linh hoạt; tăng cường cơ chế chống can thiệp tại kernel module (pmd-driver) nhằm đảm bảo tính toàn vẹn trong môi trường thực tế, bao gồm tận dụng Trusted Platform Module (TPM); bổ sung callback ở cấp kernel để giám sát thêm các sự kiện như DLL image mapping, và xây dựng cơ chế tạm thời khóa file PE trong quá trình chờ quét ở user mode. Cuối cùng, mở rộng pipeline sang các định dạng file khác ngoài PE - chẳng hạn ELF, PDF và APK - là các hướng phát triển tiềm năng khi nền tảng nghiên cứu đã được xây dựng vững chắc.

Nhìn chung, khóa luận đã có những đóng góp ở cả phương diện nghiên cứu và ứng dụng thực tiễn. Về mặt nghiên cứu, đề tài cung cấp các bằng chứng thực nghiệm cho thấy hiệu quả của phương pháp trích xuất capability trong bài toán phát hiện mã độc PE. Về mặt công nghệ, khóa luận xây dựng thành công một hệ thống bảo vệ endpoint hoàn chỉnh, kết hợp giữa lập trình hệ thống trên Windows và các kỹ thuật học máy hiện đại. Những kết quả và kinh nghiệm thu được từ đề tài được kỳ vọng sẽ trở thành nền tảng hữu ích cho các nghiên cứu và ứng dụng tiếp theo trong lĩnh vực phát hiện mã độc và an toàn thông tin.

## Tài liệu tham khảo

- [1] Kaspersky. The cyber surge: Kaspersky detected 467,000 malicious files daily in 2024. <https://www.kaspersky.com/about/press-releases/the-cyber-surge-kaspersky-detected-467000-malicious-files-daily-in-2024>, 2024.
- [2] SafeGate. Hơn 46% cơ quan, doanh nghiệp Việt Nam bị tấn công mạng trong năm 2024. <https://safegate.vn/vi/post/hon-46--co-quan--doanh-nghie--p-viet-nam-bi-ta--n-co--ng-ma--ng-trong-nam-2024-68.htm>, 2024.
- [3] StatCounter. Desktop Operating System market share worldwide - StatCounter Global Stats. <https://gs.statcounter.com/os-market-share/desktop/worldwide>, 2024.
- [4] AVTest. Malware statistics. <https://www.av-test.org/en/statistics/malware>, 2024.
- [5] Hyrum S. Anderson and Phil Roth. EMBER: An open dataset for training static PE malware machine learning models. In *arXiv preprint arXiv:1804.04637*, 2018.
- [6] Tạp chí điện tử Luật sư Việt Nam. Đặt mục tiêu đến năm 2030, Việt Nam có đội ngũ 10.000 chuyên gia an ninh mạng trình độ cao. <https://lsvn.vn/dat-muc-tieu-den-nam-2030-viet-nam-co-doi-ngu-10-000-chuyen-gia-an-ninh-mang-trinh-do-cao-a169160.html>, 2026.
- [7] Wikipedia. Malware. <https://en.wikipedia.org/wiki/Malware>, 2026.
- [8] Wikipedia. Advanced persistent threat. [https://en.wikipedia.org/wiki/Advanced\\_persistent\\_threat](https://en.wikipedia.org/wiki/Advanced_persistent_threat), 2026.
- [9] Wikipedia. Endpoint security. [https://en.wikipedia.org/wiki/Endpoint\\_security](https://en.wikipedia.org/wiki/Endpoint_security), 2025.
- [10] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer's Manual*, 2021.

- [11] Advanced Micro Devices. *AMD64 Architecture Programmer's Manual, Volume 2: System Programming*, 2020.
- [12] Microsoft. x64 Calling Convention. <https://learn.microsoft.com/en-us/cpp/build/x64-calling-convention?view=msvc-170>, 2025.
- [13] Microsoft. Portable Executable (PE) and COFF Specification. <https://learn.microsoft.com/en-us/windows/win32/debug/pe-format>, 2025.
- [14] 0xRick's Blog. (7-part series) A dive into the PE file format - PE file structure. <https://0xrick.github.io/win-internals/pe1>, 2025.
- [15] Jagsir Singh and Jaswinder Singh. A survey on machine learning-based malware detection in executable files. *Journal of Systems Architecture*, 112:101861, 2021.
- [16] A. Damodaran, F. D. Troia, C. A. Visaggio, et al. A comparison of static, dynamic, and hybrid analysis for malware detection. *Journal of Computer Virology and Hacking Techniques*, 13:1–12, 2017.
- [17] Robert J. Joyce, Gideon Miller, Phil Roth, Richard Zak, Elliott Zaresky-Williams, Hyrum Anderson, Edward Raff, and James Holt. EMBER2024 - A Benchmark Dataset for Holistic Evaluation of Malware Classifier. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2025.
- [18] VirusShare. VirusShare - because sharing is caring. <https://virusshare.com>, 2024.
- [19] VirusTotal. VirusTotal - free online virus, malware and URL scanner. <https://www.virustotal.com>, 2024.
- [20] Richard Harang and Ethan M. Rudd. SOREL-20M: A large scale benchmark dataset for malicious PE detection. In *arXiv preprint arXiv:2012.07634*, 2020.
- [21] Ian T. Jolliffe. *Principal Component Analysis*. Springer, 2nd edition, 2002.
- [22] Nathan Halko, Per-Gunnar Martinsson, and Joel A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011.

- [23] Omer Sagi and Lior Rokach. Ensemble learning: A survey. *WIREs Data Mining and Knowledge Discovery*, 8(4):e1249, 2018.
- [24] Mandiant. capa: The flare team’s open-source tool to identify capabilities in executable files. <https://github.com/mandiant/capa>, 2020.
- [25] MITRE. MITRE ATT&CK: Adversarial tactics, techniques, and common knowledge. <https://attack.mitre.org>, 2024.
- [26] MITRE. Malware behavior catalog (MBC). <https://github.com/MBCProject/mbc-markdown>, 2024.
- [27] Jinsung Kim, Younghoon Ban, Eunbyeol Ko, Haehyun Cho, and Jeong Hyun Yi. Mapas: a practical deep learning-based android malware detection system. *International Journal of Information Security*, 21(4):725–738, 2022.
- [28] Zhenglin Li, Haibei Zhu, Houze Liu, Jintong Song, and Qishuo Cheng. Comprehensive evaluation of mal-api-2019 dataset by machine learning in malware detection. *arXiv preprint arXiv:2403.02232*, 2024.
- [29] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [30] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. “Why Should I Trust You?”: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 1135–1144, 2016.
- [31] Luca Demetrio, Battista Biggio, Giovanni Lagorio, Fabio Roli, and Alessandro Armando. Explaining vulnerabilities of deep learning to adversarial malware binaries. In *Proceedings of the Italian Conference on Cybersecurity (ITASEC)*, 2019.
- [32] Microsoft. Windows API index. <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>, 2024.
- [33] Microsoft. Kernel-mode driver framework (KMDF). <https://learn.microsoft.com/en-us/windows-hardware/drivers/wdf/>, 2024.
- [34] Sebastián Ramírez. FastAPI. <https://fastapi.tiangolo.com>, 2024.



- [35] Docker Inc. Docker: Accelerated container application development. <https://www.docker.com>, 2024.
- [36] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [37] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, et al. Scikit-learn: Machine learning in python, 2011. *Journal of Machine Learning Research*.